# Université de Djibouti
## Faculté Sciences


Thème :

# Parking Intelligent


Encadré par : **Moubarik Barre**

**Membres du groupe :**

1. simen Abdourahman Hassan
2. Sihan Abdourahman Mohamed
3. Soumeya Bachir Mahamoud
4. Salma Aden


Année académique :

**2025 – 2026**

# Table des matières

Remerciements ..... 5
Résumé ..... 6
Abstract ..... 6
**Contexte et motivation** ..... 7
**Objectifs du projet** ..... 7
**Défis techniques rencontrés** ..... 8
**Structure du rapport** ..... 8
I.1 Contexte général ..... 9
I.2 Problématique du stationnement à Djibouti ..... 10
I.3 Présentation de la solution : MOUBARIK Parking ..... 10
I.4 Acteurs du système ..... 11
I.4.1 Le Conducteur ..... 11
I.4.2 L'Administrateur ..... 11
I.4.3 Le Capteur (acteur système) ..... 11
I.4.4 Le Système de Paiement (acteur externe) ..... 12
I.5 Périmètre et limites du projet ..... 12
II.1 Étude de l'existant ..... 13
II.2 Critique de l'existant et positionnement de notre solution ..... 13
II.3 Analyse des besoins ..... 14
II.3.1 Besoins fonctionnels — Conducteur ..... 14
II.3.2 Besoins fonctionnels — Administrateur ..... 15
II.4 Besoins non fonctionnels ..... 15
II.5 Diagramme des cas d'utilisation (UML) ..... 16
**III — Méthodologie de Gestion du Projet** ..... 18
III.2 Organisation de l'équipe (Rôles Scrum) ..... 18
III.3 Product Backlog ..... 19
III.4 Planification des Sprints (Diagramme de Gantt) ..... 19
III.5 Déroulement des Sprints ..... 20
IV.1 Architecture générale du système ..... 20
**IV.2 Diagrammes UML** ..... 21
**IV.2.1 Diagramme de classes** ..... 21
**IV.2.3 Diagramme de séquence** ..... 23
**IV.2.4 Diagramme d'activité** ..... 23
**IV.2.5 Diagramme de d'état** ..... 24
**IV.2.6 Diagramme de Collaboration** ..... 26

V .2 Technologies Utilisées ..... 28
V .2.1 React.js — Framework Frontend ..... 28
**V .2.2 PHP — Backend et API REST** ..... 28
**V.2.3 MySQL — Base de Données** ..... 29
**V.2.4 Bibliothèques complémentaires** ..... 29
Conclusion de la section ..... 30
5.3 Présentation des Fonctionnalités Principales ..... 30
V.3.1 Recherche de place disponible ..... 30
**V.3.2 Réservation de place** ..... 31
**V.3.3 Navigation vers le parking** ..... 31
**V.3.4 Paiement en ligne** ..... 31
**V.3.5 Historique des réservations** ..... 32
Conclusion de la section ..... 32
V.4 Captures d'écran de l'application ..... 33
V.4.1 Interfaces Publiques ..... 33
**Page d'Accueil (Route : /)** ..... 33
**Page de Connexion (Route : /connexion)** ..... 35
**Page d'Inscription (Route : /inscription)** ..... 35
V.4.2 Interface Conducteur ..... 36
**Dashboard Conducteur (Route : /dashboard)** ..... 36
**Plan des Places (Route : /plan)** ..... 37
**Réservation (Route : /reservation)** ..... 37
**Profil Conducteur (Route : /profil)** ..... 38
V.4.3 Interface Administrateur ..... 39
**Tableau de Bord Administrateur (Route : /admin)** ..... 39
**Gestion des Places (Route : /admin/places)** ..... 39
**Gestion des Tarifs (Route : /admin/tarifs)** ..... 40
**Gestion des Paiements (Route : /admin/paiements)** ..... 41
Conclusion de la section ..... 42
VI . Tests ,Déploiement et Maintenance ..... 42
VI .1 Stratégie de Test ..... 42
VI.2 Tests Fonctionnels ..... 43
**Authentification** ..... 43
**Réservation** ..... 43
**Paiement** ..... 43
**Administration** ..... 43

VI.3 Tests d'Interface ..... 44
**Compatibilité et réactivité** ..... 44
**Thème clair / sombre** ..... 44
**Performance** ..... 44
VI.4 Résultats et Corrections ..... 44
**Bilan des tests** ..... 44
**Corrections apportées** ..... 44
**Limites et recommandations** ..... 45
VI.5 Déploiement ..... 45
VI.5.1 Déploiement en environnement de développement local ..... 45
VI.5.2 Déploiement en environnement de production ..... 46
Conclusion de la section ..... 46
VI.6 Maintenance Logiciel ..... 47
VI.6.1 Maintenance corrective ..... 47
VI.6.2 Maintenance evolutive ..... 47
VI.6.3 Maintenance adaptative ..... 48
Conclusion de la section ..... 48

CONCLUSION GENERALE ..... 49

BILAN DU PROJET ..... 49

Difficultés rencontrées ..... 50

Perspectives et améliorations futures ..... 50

Bibliographie/ webographie ..... 51

Annexes ..... 52

**Annexe A — Tableau de suivi global des tests ..... 52**

**Annexe B — Script SQL de création de la base de données ..... 53**

# Remerciements

Au terme de ce travail, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos remerciements les plus sincères vont en premier lieu à notre encadrant, le Docteur **Moubarek Barré Hassan**, dont la rigueur intellectuelle, la disponibilité et les précieux conseils nous ont guidés à chaque étape de ce parcours. Votre confiance en notre équipe a été un moteur essentiel.

Nous adressons également toute notre reconnaissance à l'Université de Djibouti, à la Faculté des Sciences et à l'ensemble du corps enseignant qui ont su nous transmettre les compétences techniques et intellectuelles nécessaires à l'accomplissement de ce projet.

Une pensée particulière va à nos familles respectives, piliers silencieux mais indispensables. Votre soutien inconditionnel, votre patience dans les moments de doute et votre encouragement constant nous ont portées jusqu'au bout.

À nos amies et camarades de promotion qui ont partagé cette aventure avec nous — les longues nuits de débogage, les moments de découragement surmontés, les victoires célébrées ensemble — merci pour cette solidarité si précieuse.

*Enfin, ce rapport n'est pas seulement la conclusion d'une licence — il est le reflet de quatre années d'apprentissage, d'efforts partagés et d'une équipe soudée qui a su transformer une idée en une solution concrète au service de la ville de Djibouti.*

# Résumé

Ce projet consiste en la conception et le développement d'une application web de gestion intelligente du stationnement, baptisée MOUBARIK Parking, dédiée à la ville de Djibouti. Face à la croissance du parc automobile et à l'absence de solutions numériques locales, cette plateforme permet aux conducteurs de consulter en temps réel la disponibilité des places, d'effectuer des réservations en ligne et de gérer leurs sessions de stationnement via une interface intuitive.

L'application repose sur une architecture client-serveur : le frontend est développé en React.js avec React Router, tandis que le backend est assuré par une API REST en PHP communiquant avec une base de données MySQL. La gestion des utilisateurs distingue deux rôles : les conducteurs (réservation, signalement d'arrivée, paiement, historique) et les administrateurs (supervision, rapports, gestion des tarifs, confirmation des paiements).

# Abstract

This project involves the design and development of a smart parking management web application called MOUBARIK Parking, dedicated to the city of Djibouti. In response to the growth of the car fleet and the absence of local digital solutions, the platform allows drivers to check real-time parking availability, make online reservations, and manage their parking sessions through an intuitive interface.

The application relies on a client-server architecture: the frontend is built with React.js and React Router, while the backend is handled by a PHP REST API communicating with a MySQL database. User management distinguishes two roles: drivers (reservation, arrival, payment, history) and administrators (supervision, reports, pricing, payment confirmation).

# Introduction Générale

Choisir un sujet de projet de fin d'études, c'est souvent choisir une réponse à un problème que l'on a soi-même vécu ou observé. C'est précisément dans cet esprit que nous avons orienté notre réflexion vers la problématique du stationnement urbain à Djibouti — non pas parce que ce sujet figurait dans une liste de thèmes suggérés, mais parce que la réalité de notre quotidien nous y a conduits naturellement.

## Contexte et motivation

Quiconque circule régulièrement dans la capitale djiboutienne a été confronté, à un moment ou un autre, à la difficulté de trouver une place de stationnement. Aux heures de pointe, aux abords des administrations, des marchés ou des zones commerciales, la scène est familière : des véhicules qui tournent en rond, des conducteurs qui cherchent à tâtons une place libre, parfois pendant de longues minutes. Ce temps perdu n'est pas anodin — il génère de la frustration, alimente la congestion et contribue inutilement à la pollution urbaine.

Ce qui nous a frappées, c'est l'absence totale de solution numérique locale pour répondre à ce problème. Contrairement à de nombreuses capitales africaines et mondiales qui disposent déjà d'applications de gestion de stationnement, Djibouti n'en possède aucune à ce jour. Les quelques parkings existants fonctionnent encore de manière entièrement manuelle : ni affichage de disponibilité, ni réservation à l'avance, ni suivi des sessions. Le conducteur arrive, espère, et repart parfois bredouille.

C'est ce constat — simple, concret, ancré dans notre quotidien — qui a constitué le point de départ de ce projet. Nous avons voulu concevoir non pas une application théorique, mais un système réellement utile, pensé pour les contraintes et les usages spécifiques de Djibouti.

## Objectifs du projet

Notre objectif principal était de développer une plateforme web complète de gestion intelligente du stationnement, baptisée MOUBARIK Parking, qui permette de digitaliser entièrement le cycle de vie d'une session de stationnement — de la recherche de place jusqu'au paiement. Plus précisément, nous avons visé à :

- Offrir aux conducteurs une visibilité en temps réel sur la disponibilité des places, sans avoir à se déplacer physiquement.
- Permettre la réservation anticipée d'une place, avec confirmation instantanée et gestion automatique de l'expiration.
- Assurer un suivi fiable et transparent de chaque session de stationnement, du signalement d'arrivée jusqu'au règlement final.

- Mettre à disposition des administrateurs des outils de supervision, d'analyse et de gestion tarifaire adaptés à la réalité opérationnelle d'un parking djiboutien.
- Concevoir une architecture technique robuste, sécurisée et évolutive, capable de s'adapter à une montée en charge future.

## Défis techniques rencontrés

Si la vision du projet était claire dès le départ, sa réalisation technique nous a confrontées à des défis que nous n'avions pas entièrement anticipés. Les deux plus significatifs ont été la synchronisation en temps réel des statuts de places et la gestion cohérente du cycle de paiement.

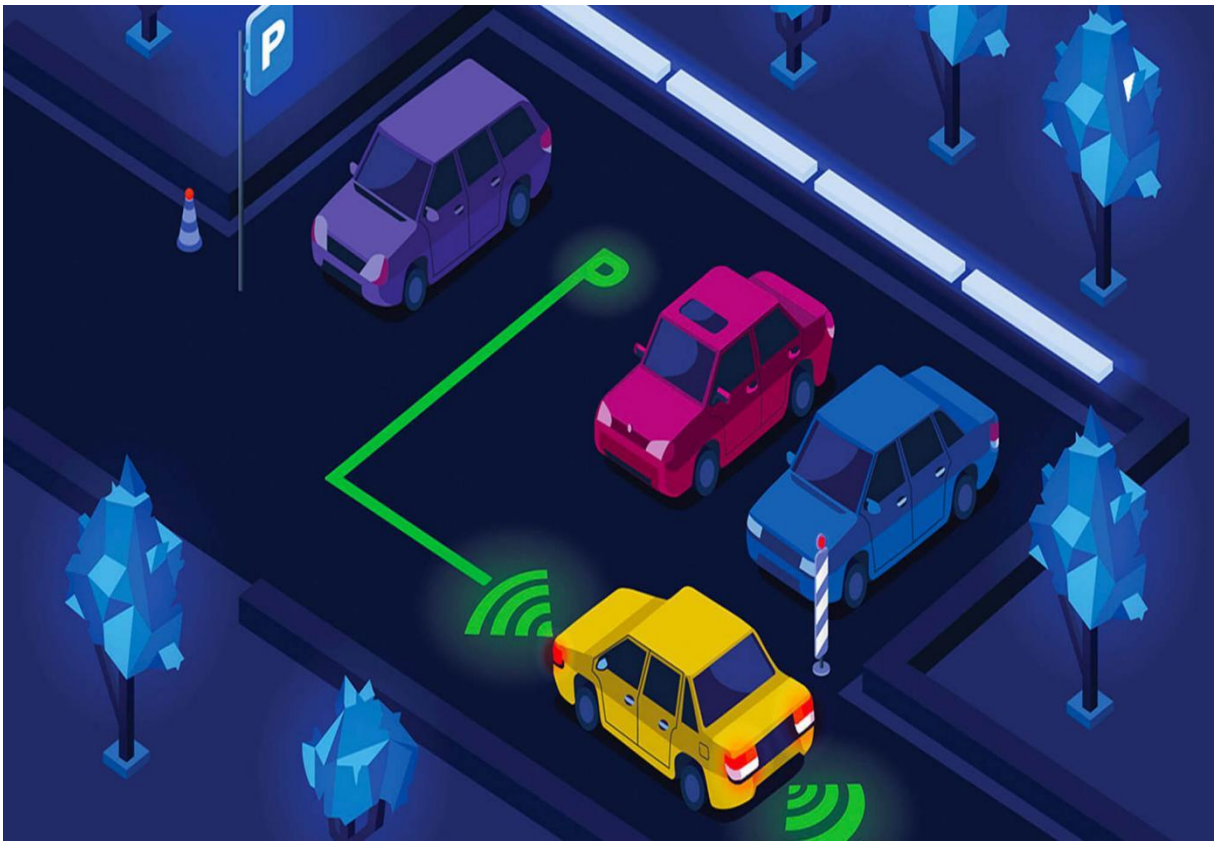
La synchronisation temps réel est au cœur de l'utilité de notre système : une information erronée sur la disponibilité d'une place — même avec quelques secondes de décalage — peut rendre l'ensemble du système peu fiable aux yeux des utilisateurs. Nous avons donc dû concevoir un mécanisme côté serveur, la fonction syncPlaceStatut(), capable de réconcilier en permanence l'état réel des places avec les réservations actives en base de données, tout en gérant les cas limites tels que l'expiration automatique des réservations non confirmées au bout de quinze minutes.

La gestion du paiement a également nécessité une attention particulière. Nous souhaitions que le calcul du montant soit dynamique, basé sur la durée réelle de la session, et que la confirmation du paiement soit robuste même en cas d'interruption de connexion. Ces contraintes nous ont amenées à réfléchir soigneusement à la cohérence transactionnelle de notre API, en recourant aux transactions PDO avec mécanisme de rollback pour garantir l'intégrité des données en toutes circonstances.

## Structure du rapport

Ce rapport rend compte de l'ensemble de notre démarche, depuis l'analyse du problème jusqu'à la livraison du produit final. Il est organisé en six PARTIE complémentaires. LE PARTIE I situe le projet dans son contexte et présente la solution développée. Le PAR*ETI* II expose l'étude de l'existant et la définition des besoins. Le PARTIE III détaille notre approche méthodologique, fondée sur la méthode Agile/Scrum. Le PARTIE IV présente les choix de conception — architecture, modèle de données et diagrammes UML. Le PARTIE V décrit la réalisation technique et les fonctionnalités implémentées. Enfin, le PARTIE VI présente les tests effectués et les résultats obtenus.

# I. Présentation du Projet




## I.1 Contexte général

Djibouti est une ville en mutation. Petite par la superficie mais dense par les flux — humains, commerciaux, diplomatiques — la capitale djiboutienne concentre sur quelques kilomètres carrés l'essentiel de l'activité du pays. Cette densité, combinée à une motorisation croissante des ménages, a transformé la question du stationnement en un enjeu urbain réel et quotidien.

À l'échelle mondiale, les villes répondent à ce défi par le développement de systèmes de parking intelligents — des infrastructures connectées qui permettent aux conducteurs de localiser, réserver et payer leur stationnement depuis leur téléphone ou leur navigateur. Ces systèmes existent depuis plusieurs années en Europe, en Asie et dans de nombreuses métropoles africaines. Djibouti, elle, n'en possède encore aucun.

C'est dans cet écart entre le besoin évident et l'absence de réponse numérique locale que s'inscrit notre projet. MOUBARIK Parking ne cherche pas à reproduire ce qui existe ailleurs — il cherche à proposer une première réponse adaptée, conçue par des étudiantes djiboutiennes pour leur propre ville.

## I.2 Problématique du stationnement à Djibouti

Pour cadrer notre réflexion, nous avons cherché à décomposer le problème du stationnement à Djibouti en plusieurs dimensions concrètes. Cette analyse nous a permis d'identifier cinq points de friction principaux, qui ont directement orienté les fonctionnalités que nous avons décidé de développer :

- L'invisibilité des places disponibles : un conducteur ne peut pas savoir, avant de se déplacer, si une place l'attend à destination. Cette incertitude engendre des déplacements inutiles et une perte de temps considérable.
- L'impossibilité de réserver à l'avance : même lorsqu'un parking est connu, il est impossible d'en garantir une place. Le premier arrivé est le premier servi, ce qui pénalise ceux qui planifient leurs déplacements.
- La gestion manuelle des entrées et des paiements : dans les parkings existants, tout repose sur la présence physique d'un gardien, ce qui limite les horaires d'ouverture et expose à des erreurs ou des irrégularités.
- L'absence de données sur l'utilisation : aucun gestionnaire de parking à Djibouti ne dispose aujourd'hui d'un tableau de bord lui indiquant son taux d'occupation, ses heures de pointe ou ses revenus mensuels.
- La fracture numérique entre Djibouti et les standards internationaux : pendant que le reste du monde digitalise le stationnement, Djibouti reste en retrait, faute de solutions locales adaptées à son contexte.

## I.3 Présentation de la solution : MOUBARIK Parking

MOUBARIK Parking est une application web développée en React.js côté client et PHP côté serveur, avec une base de données MySQL. Son nom rend hommage à notre encadrant, le Dr. Moubarek Barré Hassan, dont l'accompagnement a été déterminant dans l'aboutissement de ce projet.

L'application repose sur une architecture client-serveur découplée : le frontend React communique avec une API REST PHP via des requêtes HTTP authentifiées par token Bearer. Cette séparation claire des responsabilités nous a permis de développer et tester les deux couches indépendamment, et garantit une meilleure évolutivité à long terme.

La particularité de notre approche réside dans le niveau de détail que nous avons apporté à chaque fonctionnalité. Par exemple, le statut d'une place de parking n'est pas un simple booléen — il peut prendre trois valeurs distinctes (available, reserved, occupied) qui sont synchronisées en temps réel par un mécanisme côté serveur. De même, une réservation non confirmée expire automatiquement après quinze minutes, libérant la place pour d'autres conducteurs sans intervention humaine.

| | |
|---|---|
| Nom du projet | MOUBARIK Parking |
| Type | Application web — Gestion intelligente de stationnement |
| Frontend | React.js, React Router v6, Lucide Icons, CSS personnalisé |
| Backend | PHP 8 — API REST, PDO, Transactions MySQL |
| Base de données | MySQL (WAMP) — Tables : Place, Reservation, Etablissement, Tarif... |
| Authentification | Token Bearer (Authorization Header), rôles : conducteur / admin |
| Acteurs du système | Conducteur, Administrateur, Capteur (simulé), Système de paiement |
| Périmètre géographique | Ville de Djibouti — République de Djibouti |
| Contexte académique | Projet de Geni Logiceil— Licence Informatique L3, 2025–2026 |
| Encadrant | Dr. Moubarek Barré Hassan — Faculté des Sciences, Université de Djibouti |

**Tableau 1 — Fiche d'identité du projet MOUBARIK Parking**

## I.4 Acteurs du système

L'identification précise des acteurs a été l'une de nos premières étapes de conception. Contrairement à ce que l'on pourrait imaginer, le système ne se limite pas à deux acteurs humains — il implique également deux acteurs systèmes qui jouent un rôle essentiel dans le fonctionnement de l'application.

### I.4.1 Le Conducteur

Le conducteur est l'utilisateur final, celui pour qui l'application a été pensée en premier lieu. Il s'inscrit avec son adresse email, se connecte, et peut dès lors accéder à l'ensemble des fonctionnalités de réservation. Son parcours type dans l'application suit un cycle logique : consulter les parkings disponibles, visualiser les places libres, réserver, se déplacer, signaler son arrivée, stationner, terminer sa session et payer. À chaque étape, l'interface lui fournit un retour immédiat sur l'état de sa réservation.

### I.4.2 L'Administrateur

L'administrateur est le gestionnaire du parking. Son interface est entièrement différente de celle du conducteur — il accède à un tableau de bord de supervision qui lui permet de visualiser en temps réel l'état de chaque place, de consulter les réservations en cours, de définir les tarifs et de valider les paiements. Par souci de sécurité, les comptes administrateurs ne sont pas créés via une interface publique : ils sont provisionnés directement en base de données, ce qui évite toute tentative d'usurpation de rôle.

### I.4.3 Le Capteur (acteur système)

Dans une version industrialisée du système, des capteurs physiques installés sur chaque place de parking transmettraient automatiquement leur état au serveur. Dans notre implémentation académique, ce rôle est simulé : la mise à jour des statuts de places est déclenchée soit par les actions des conducteurs (réservation, arrivée, fin de session), soit par l'administrateur. L'architecture a néanmoins été conçue pour accueillir une intégration IoT réelle sans modification structurelle majeure.

### I.4.4 Le Système de Paiement (acteur externe)

Le système de paiement représente la passerelle financière externe. Dans notre cas, il est modélisé comme un acteur à part entière dans le diagramme UML car il intervient activement dans deux cas d'utilisation clés : le traitement du paiement en fin de session et l'envoi de la notification de confirmation. Cette séparation architecturale permet d'envisager facilement l'intégration future d'un prestataire de paiement réel (mobile money, carte bancaire).

## I.5 Périmètre et limites du projet

Tout projet académique doit être honnête sur ce qu'il couvre et ce qu'il laisse de côté. MOUBARIK Parking implémente les fonctionnalités suivantes :

- Authentification complète avec gestion différenciée des rôles conducteur et administrateur.
- Consultation en temps réel des places disponibles, avec synchronisation automatique des statuts (available / reserved / occupied).
- Réservation de place avec blocage temporaire de quinze minutes et libération automatique en cas de non-confirmation.
- Gestion complète du cycle de session : réservation → signalement d'arrivée → session active → fin de session → paiement.
- Interface d'administration avec tableau de bord, gestion des tarifs, validation des paiements et génération de rapports.
- Système de notifications intégré et module de collecte d'avis conducteurs.

En revanche, certains aspects ont volontairement été mis hors périmètre, faute de temps ou d'infrastructure :

- L'intégration de capteurs IoT physiques pour la détection automatique de présence de véhicules.
- Le raccordement à une vraie passerelle de paiement en ligne (Mobile Money Djibouti Telecom, carte bancaire).
- Le développement d'une application mobile native pour iOS et Android.

*Ces limites ne constituent pas des faiblesses du projet — elles définissent clairement le périmètre de ce qui a été réalisé et tracent*

# II. Étude de l'Existant et Analyse des Besoins

## II.1 Étude de l'existant

Avant d'écrire la moindre ligne de code, nous avons consacré du temps à comprendre ce qui existait déjà. Cette étape est souvent sous-estimée dans les projets académiques — pourtant, c'est elle qui permet de justifier les choix que l'on fait et d'éviter de réinventer ce qui fonctionne déjà ailleurs. Notre étude a porté sur quatre applications internationales représentatives du secteur.

*Il est important de noter que nous n'avons pas pu tester ces applications directement à Djibouti — aucune d'entre elles n'y est disponible. Notre analyse s'est donc appuyée sur la documentation officielle, les avis d'utilisateurs en ligne et les articles académiques disponibles sur le sujet.*

| | | | |
|---|---|---|---|
| **EasyPark (Suède)** | Recherche de parkings, paiement mobile, extension de durée, zones partenaires | Interface rapide, présence dans 25+ pays, paiement sans ticket physique | Indisponible en Afrique subsaharienne et à Djibouti, ne couvre pas le cycle complet arrivée/départ |
| **ParkWhiz (USA)** | Réservation anticipée, passe mensuel, intégration Google Maps, QR code | Fiabilité de la réservation, bon historique, partenariats avec grands parkings | Limité aux villes américaines, pas d'interface admin pour gestionnaires locaux |
| **Parkopedia (UK)** | Comparateur de parkings, tarifs, navigation GPS, données temps réel | Base de données mondiale, compatible GPS embarqué | Aucune réservation directe, données parfois obsolètes dans les pays émergents |
| **PayByPhone (Canada)** | Paiement smartphone, rappels expiration, prolongation à distance, historique | Rapidité, intégration apps navigation, large réseau villes partenaires | Pas de réservation préalable, dépend d'accords avec collectivités locales |

**Tableau 2 — Comparaison des principales solutions de parking existantes**

## II.2 Critique de l'existant et positionnement de notre solution

L'analyse de ces quatre solutions nous a permis de dégager un constat central : il n'existe pas, à ce jour, de solution de gestion de parking conçue pour et adaptée au contexte djiboutien. Les applications existantes sont toutes pensées pour des marchés occidentaux matures, avec des infrastructures qui n'existent pas encore à Djibouti sous la même forme.

De ce constat émergent plusieurs lacunes spécifiques que notre projet a cherché à combler :

| | | |
|---|---|---|
| **Aucune solution disponible localement** | Les conducteurs djiboutiens n'ont accès à aucun outil numérique de stationnement | Application conçue localement, en français, pour Djibouti |
| **Pas de gestion du cycle complet de session** | Les solutions existantes couvrent soit la réservation, soit le paiement — rarement les deux | Cycle complet : réservation → arrivée → session → paiement, géré de bout en bout |
| **Absence d'interface administrateur locale** | Aucun gestionnaire de parking djiboutien ne peut superviser son activité numériquement | Tableau de bord admin dédié : statuts, tarifs, rapports, validation paiements |
| **Données non synchronisées en temps réel** | Un conducteur peut réserver une place déjà occupée dans les systèmes non synchronisés | Fonction syncPlaceStatut() côté serveur, synchronisation à chaque requête |
| **Pas d'expiration automatique des réservations** | Les places bloquées mais non utilisées restent indisponibles indéfiniment | Libération automatique après 15 min si la réservation n'est pas confirmée |

**Tableau 3 — Lacunes de l'existant et réponses apportées par MOUBARIK Parking**

## II.3 Analyse des besoins

La définition des besoins traduit le problème identifié en exigences concrètes pour le système. Nous avons distingué les besoins fonctionnels (ce que le système doit faire) et les besoins non fonctionnels (comment il doit le faire).

### II.3.1 Besoins fonctionnels — Conducteur

Les besoins du conducteur ont été dérivés directement de son parcours utilisateur type, depuis sa première visite sur l'application jusqu'à la fin de sa session de stationnement.

| | | |
|---|---|---|
| **BF01** | **Inscription et authentification** | Le conducteur crée un compte avec email et mot de passe. Le système lui attribue un token Bearer qui sécurise toutes ses requêtes suivantes. Un mécanisme de réinitialisation de mot de passe est disponible. |
| **BF02** | **Recherche de place disponible** | Le conducteur consulte la liste des établissements, sélectionne un parking et visualise l'état de chaque place (disponible / réservée / occupée) mis à jour en temps réel via l'API. |
| **BF03** | **Réservation d'une place** | Le conducteur sélectionne une place disponible et la réserve. Le système enregistre la réservation en statut 'pending' et démarre un compteur de 15 minutes. Passé ce délai sans confirmation, la place est automatiquement libérée. |
| **BF04** | **Modification de réservation** | Le conducteur peut modifier une réservation existante (extension de durée) avant son expiration. |
| **BF05** | **Annulation de réservation** | Le conducteur peut annuler sa réservation tant qu'elle est en statut 'pending'. La place est immédiatement remise à disposition. |
| **BF06** | **Signalement d'arrivée et session** | En arrivant physiquement, le conducteur signale son arrivée. La réservation passe en 'confirmed', la place en 'occupied', et le chronomètre de session démarre. |

| | | |
|---|---|---|
| **BF07** | **Fin de session et paiement** | Le conducteur clôture sa session. Le montant est calculé automatiquement (durée réelle x tarif horaire) et soumis à validation par l'administrateur. |
| **BF08** | **Confirmation et historique** | Après validation du paiement, le conducteur reçoit une confirmation. Il peut consulter l'intégralité de son historique avec date, lieu, durée, statut et montant. |
| **BF09** | **Notifications** | Le conducteur reçoit des notifications pour les événements clés : confirmation de réservation, rappel d'expiration, validation du paiement. |
| **BF10** | **Avis et évaluation** | Le conducteur peut soumettre un avis détaillé sur son expérience après chaque session terminée. |

**Tableau 4 — Besoins fonctionnels du Conducteur**

### II.3.2 Besoins fonctionnels — Administrateur

Les besoins de l'administrateur ont été identifiés à partir de la réalité opérationnelle d'un gestionnaire de parking : que doit-il pouvoir faire au quotidien pour gérer efficacement son infrastructure ?

| | | |
|---|---|---|
| **BA01** | **Connexion sécurisée dédiée** | L'administrateur se connecte via l'interface commune mais en sélectionnant le rôle 'admin'. Son compte est provisionné directement en base de données — il n'existe pas de page d'inscription publique pour ce rôle, par mesure de sécurité. |
| **BA02** | **Gestion des places** | L'administrateur peut ajouter de nouvelles places (numéro, localisation, étage, établissement), modifier leurs informations et les supprimer. Il peut également forcer un changement de statut si nécessaire. |
| **BA03** | **Consultation des réservations** | L'administrateur visualise en temps réel toutes les réservations en cours et consulte l'historique complet par établissement, par date ou par statut. |
| **BA04** | **Gestion des tarifs** | L'administrateur définit et ajuste les tarifs horaires par établissement. Il peut créer plusieurs niveaux tarifaires (heure creuse, heure de pointe, forfait journée). |
| **BA05** | **Validation des paiements** | L'administrateur consulte les paiements en attente et les valide manuellement, déclenchant la notification de confirmation côté conducteur. |
| **BA06** | **Rapports d'activité** | L'administrateur génère des rapports sur l'activité : taux d'occupation par période, revenus cumulés, nombre de sessions, durée moyenne de stationnement. |
| **BA07** | **Notifications et avis** | L'administrateur peut envoyer des notifications ciblées aux conducteurs et consulter les avis déposés après les sessions pour améliorer la qualité du service. |

**Tableau 5 — Besoins fonctionnels de l'Administrateur**

## II.4 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité que notre système doit respecter. Une application qui fait ce qu'elle est censée faire, mais de manière lente, peu sécurisée ou difficile à maintenir, est une application qui échoue à long terme.

| | | |
|---|---|---|
| **Performance** | Réponse API<br>en moins de 2 secondes pour 95% des requêtes | Requêtes SQL indexées sur idPlace et idReservation ; syncPlaceStatut() exécutée en une seule passe avec des clauses IN groupées plutôt qu'en boucle unitaire. |
| **Cohérence des données** | Aucune incohérence entre l'état affiché et l'état réel des places | Transactions PDO avec BEGIN/COMMIT/ROLLBACK sur toutes les opérations critiques. En cas d'erreur partielle, aucune modification n'est persistée. |
| **Sécurité** | Accès protégé et actions sensibles authentifiées | Token Bearer transmis dans l'en-tête Authorization de chaque requête. Vérification systématique côté serveur avant toute action sur une réservation ou un paiement. |
| **Fiabilité** | Libération automatique des ressources bloquées inutilement | Les réservations 'pending' depuis plus de 15 minutes sont annulées et les places libérées automatiquement lors de chaque appel à l'API des places. |
| **Utilisabilité** | Interface compréhensible sans formation préalable | Navigation par onglets clairs (Conducteur / Admin), messages d'erreur explicites en français, retours visuels immédiats sur chaque action. |
| **Maintenabilité** | Code structuré pour faciliter les évolutions futures | Composants React réutilisables, contexte d'authentification centralisé (AuthContext), backend PHP avec actions distinctes et fonctions utilitaires isolées. |
| **Compatibilité** | Fonctionnel sur les navigateurs les plus utilisés | Application React.js testée sur Chrome, Firefox, Edge et Safari. Aucune dépendance spécifique à un navigateur. |
| **Extensibilité** | Architecture ouverte aux évolutions (mobile, IoT) | API REST découplée du frontend : l'ajout d'une app mobile ou d'une intégration IoT ne nécessite pas de refonte du backend. |

**Tableau 6 — Besoins non fonctionnels, exigences et solutions techniques retenues**

## II.5 Diagramme des cas d'utilisation (UML)

Le diagramme des cas d'utilisation ci-dessous synthétise graphiquement l'ensemble des interactions identifiées entre les quatre acteurs du système et les fonctionnalités de l'application. Il constitue le document de référence qui a guidé notre conception.

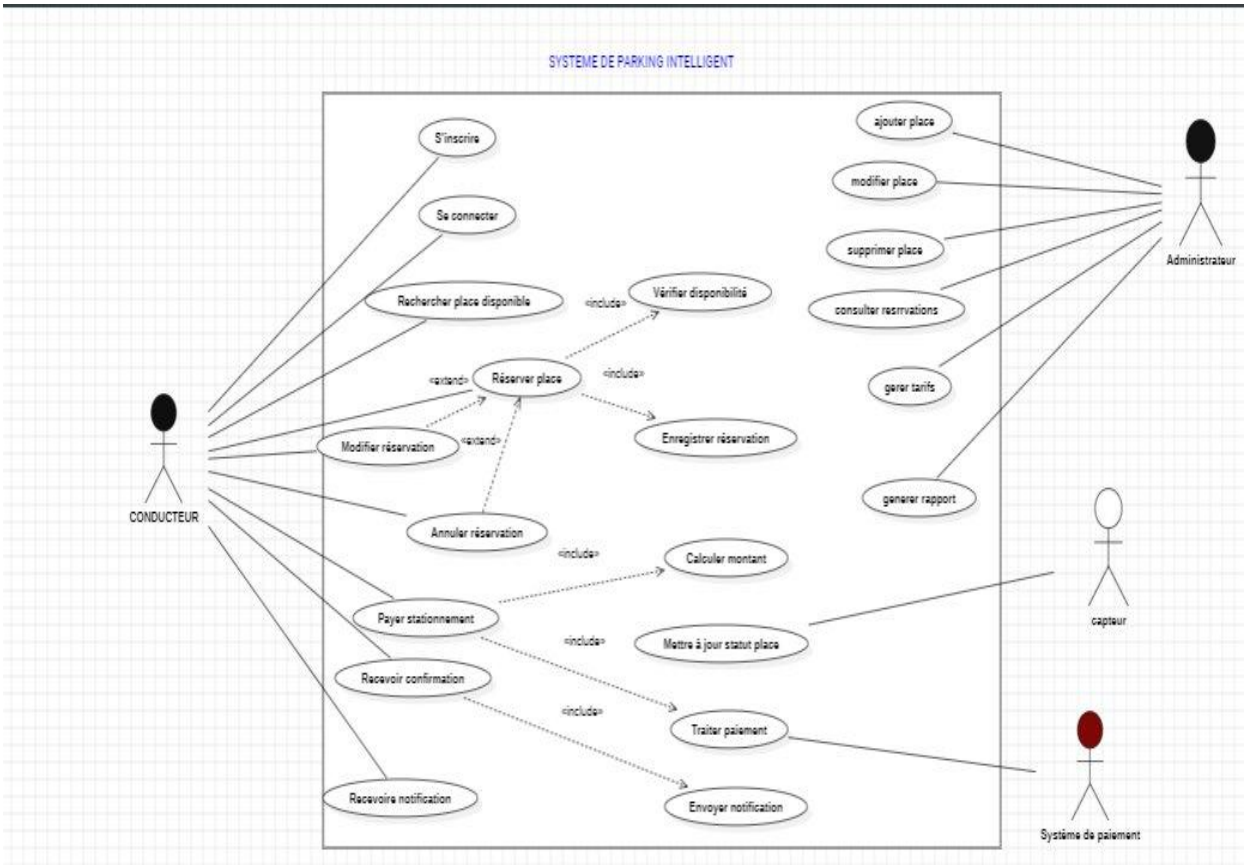



*Figure 5 — Diagramme des cas d'utilisation — Système MOUBARIK Parking*

On peut lire ce diagramme en suivant chacun des quatre acteurs. Le Conducteur interagit avec dix cas d'utilisation distincts, depuis l'inscription jusqu'à la réception de notifications, en passant par le cycle complet de réservation. Plusieurs relations <<include>> modélisent des sous-processus obligatoires : 'Réserver une place' inclut systématiquement 'Vérifier disponibilité' et 'Enregistrer réservation'.

L'Administrateur gère la dimension opérationnelle : ajout, modification et suppression de places, consultation des réservations, gestion des tarifs et rapports. Le Capteur intervient comme acteur système pour la mise à jour du statut des places. Le Système de paiement traite les transactions financières et déclenche les notifications de confirmation vers le conducteur.

# III — Méthodologie de Gestion du Projet

Dans le cadre de la réalisation de notre application mobile, une méthodologie de gestion de projet a été adoptée afin d'assurer une organisation efficace, une bonne coordination entre les membres de l'équipe et une livraison progressive des fonctionnalités. Le choix s'est porté sur une approche Agile, notamment la méthode Scrum, qui permet de s'adapter aux changements et d'améliorer continuellement le produit.

## III.1 Choix de la méthode (Agile / Scrum)

Le développement d'une application mobile nécessite une grande flexibilité en raison de l'évolution constante des besoins des utilisateurs. Pour répondre à ces exigences, la méthodologie Agile a été choisie.

Le framework Scrum a été adopté car il permet :

- un développement progressif de l'application
- une adaptation rapide aux changements
- une meilleure communication au sein de l'équipe
- une livraison régulière de versions fonctionnelles

Le projet est ainsi découpé en plusieurs itérations appelées *Sprints*, permettant de développer l'application étape par étape.

## III.2 Organisation de l'équipe (Rôles Scrum)

Dans le cadre de ce projet d'application mobile, l'équipe a été organisée en s'inspirant des principes de Scrum, tout en adaptant la répartition des tâches selon les compétences de chaque membre.

L'équipe est composée de :

- Simen Abdourahman Hassan
- Sihan Abdourahman
- Soumeya Bachir
- Salma Aden

La répartition des responsabilités est la suivante :

- **Développement** : assuré par *Simen Abdourahman Hassan*, chargée de la réalisation technique de l'application mobile et de l'implémentation des fonctionnalités.
- **Conception et tests** : assurés par *Sihan Abdourahman* et *Soumeya Bachir*, responsables de la conception des interfaces, de l'expérience utilisateur et des tests de validation.
- **Analyse des besoins** : assurée par *Salma Aden*, chargée d'identifier les besoins des utilisateurs et de définir les fonctionnalités principales de l'application.

### III.3 Product Backlog

Le Product Backlog représente l'ensemble des fonctionnalités attendues dans l'application mobile. Ces fonctionnalités sont exprimées sous forme de *User Stories*.

Exemples de User Stories :

- En tant qu'utilisateur, je veux créer un compte afin d'accéder à l'application
- En tant qu'utilisateur, je veux me connecter pour accéder à mon espace personnel
- En tant qu'utilisateur, je veux consulter les informations disponibles
- En tant qu'utilisateur, je veux modifier mes informations personnelles
- En tant qu'utilisateur, je veux recevoir des notifications

Les éléments du Product Backlog sont priorisés selon :

- leur importance pour l'utilisateur
- leur valeur ajoutée
- leur complexité technique

Le backlog est évolutif et mis à jour tout au long du projet.

### III.4 Planification des Sprints (Diagramme de Gantt)

Le projet a été planifié en plusieurs Sprints d'une durée de 2 à 3 semaines.

Organisation des Sprints :

- **Sprint 1** : Analyse des besoins et conception des interfaces
- **Sprint 2** : Développement des fonctionnalités principales (inscription, connexion)
- **Sprint 3** : Ajout des fonctionnalités secondaires et amélioration de l'interface
- **Sprint 4** : Tests, correction des bugs et finalisation

Un diagramme de Gantt a été utilisé pour :

- organiser les tâches
- suivre l'avancement du projet
- respecter les délais

Presse-pa... Police Alignement Nombre Style

A1 Diagramme de Gantt — Parking Intelligent

| Diagramme de Gantt — Parking Intelligent | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Tâche** | **Responsable** | **Statut** | **Semaine 1** | **Semaine 2** | **Semaine 3** | **Semaine 4** | **Semaine 5** |
| **Analyse & Conception** | Équipe Design | Terminé ✓ | | | | | |
| **Base de données** | Dev Backend | Terminé ✓ | | | | | |
| **Backend (API)** | Dev Backend | Terminé ✓ | | | | | |
| **Application mobile** | Dev Mobile | Terminé ✓ | | | | | |
| **Tests & Déploiement** | Équipe QA | Terminé ✓ | | | | | |
| *✓ Toutes les tâches sont terminées — Projet 100% complété* | | | | | | | |

### III.5 Déroulement des Sprints

Chaque Sprint suit les étapes suivantes :

1. **Sprint Planning**
   Définition des objectifs et sélection des fonctionnalités à développer.
2. **Réunion quotidienne (Daily Scrum)**
   Chaque membre présente :
   - le travail réalisé
   - le travail à faire
   - les difficultés rencontrées
3. **Développement**
   Réalisation des fonctionnalités de l'application mobile.
4. **Tests**
   Vérification du bon fonctionnement et correction des erreurs.
5. **Sprint Review**
   Présentation des résultats obtenus.
6. **Sprint Retrospective**
   Analyse du travail pour améliorer les prochains Sprints.

# IV. CONCEPTION

## IV.1 Architecture générale du système

L'architecture du système **MOUBARIK Parking** repose sur une approche modulaire permettant d'assurer la gestion efficace des parkings intelligents. Cette architecture est basée sur une séparatio claire entre les différentes couches du système.

Le système est structuré autour de trois principales composantes :

- **La couche présentation (Frontend)** :
  Elle représente l'interface utilisateur permettant aux conducteurs et aux administrateurs d'interagir avec le système. Elle offre des fonctionnalités telles que la recherche de places, la réservation et le paiement.
- **La couche logique (Backend)** :
  Cette couche gère le traitement des données et implémente les règles métier du système. Elle assure la gestion des réservations, des paiements, des notifications et de la disponibilité des places.
- **La couche données (Base de données)** :
  Elle permet le stockage et la gestion des informations relatives aux utilisateurs, aux places de parking, aux réservations et aux paiements.

Cette architecture garantit :

- Une meilleure maintenabilité
- Une évolutivité du système
- Une séparation des responsabilités

## IV.2 Diagrammes UML

Les diagrammes UML permettent de représenter de manière standardisée les différentes composantes du système ainsi que leurs interactions. Ils facilitent la compréhension, la conception et la communication entre les acteurs du projet.

Dans ce projet, nous utilisons trois types principaux de diagrammes UML :

- Diagramme de classes (structure)
- Diagramme de séquence (interaction)
- Diagramme d'activité (comportement)

### IV.2.1 Diagramme de classes

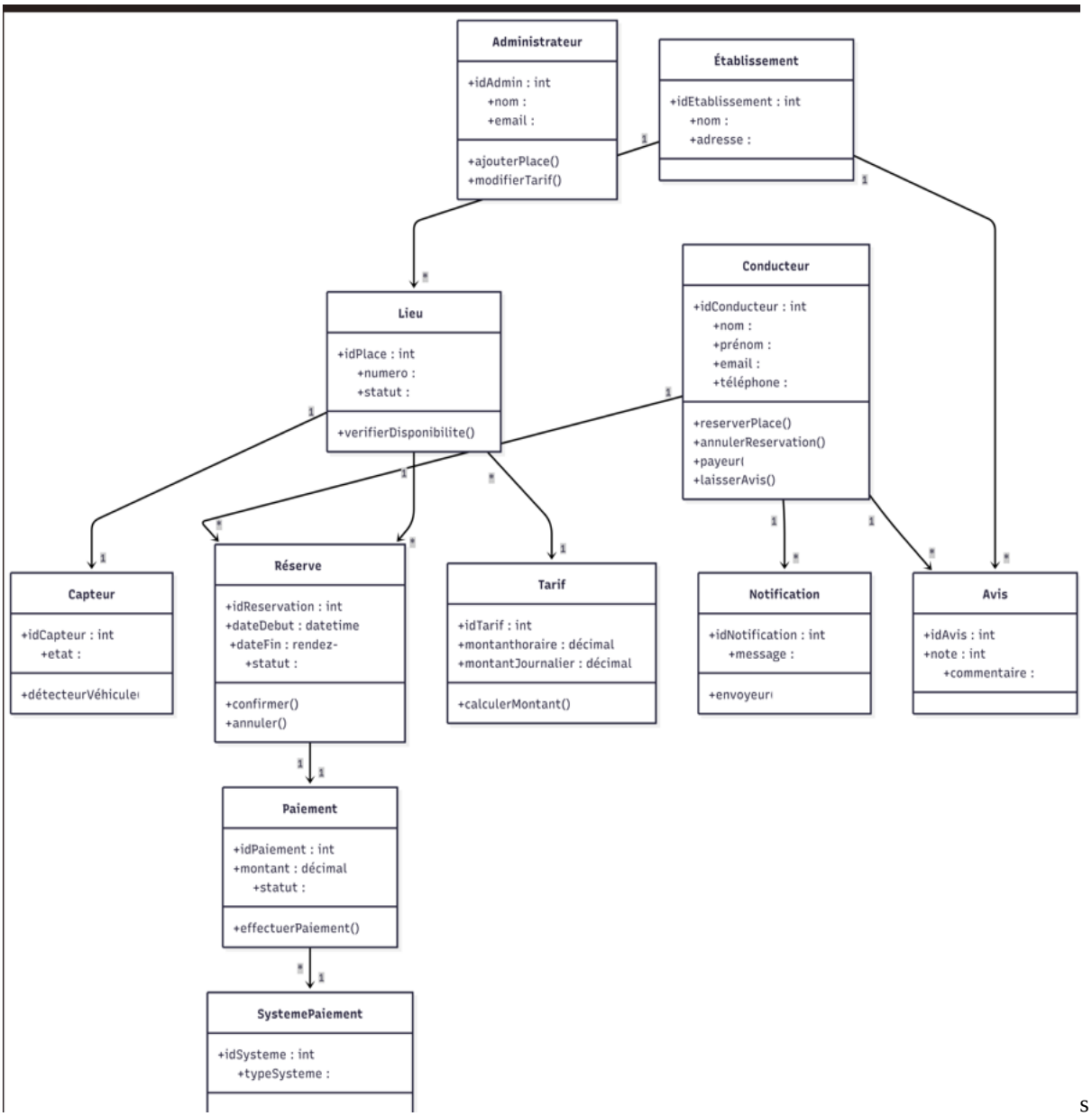
Les principales classes identifiées sont :

- **Conducteur** : représente l'utilisateur du système
- **Administrateur** : gère le système
- **Place** : représente une place de parking
- **Réservation** : permet de réserver une place
- **Paiement** : gère les transactions financières
- **Capteur** : détecte l'état des places
- **Tarif** : définit les prix
- **Etablissement** : représente les parkings
- **Notification** : informe les utilisateurs
- **Avis** : permet d'évaluer le service

Les relations entre ces classes sont définies comme suit :

- Un conducteur peut effectuer plusieurs réservations
- Une réservation concerne une seule place
- Une réservation génère un paiement
- Une place est associée à un capteur
- Un établissement contient plusieurs places

Ce diagramme permet de comprendre l'organisation globale du système.




s

Figure : diagramme de classe

### IV.2.3 Diagramme de séquence

Le diagramme de séquence décrit le déroulement des interactions entre les différents acteurs du système dans un ordre chronologique.

Dans notre cas, il illustre le processus de réservation d'une place :

1. Le conducteur recherche une place disponible
2. Le système vérifie la disponibilité
3. Une réservation est créée
4. Le conducteur effectue le paiement
5. Le système confirme la réservation

Ce diagramme met en évidence la communication entre les composants du système et le flux des opérations.

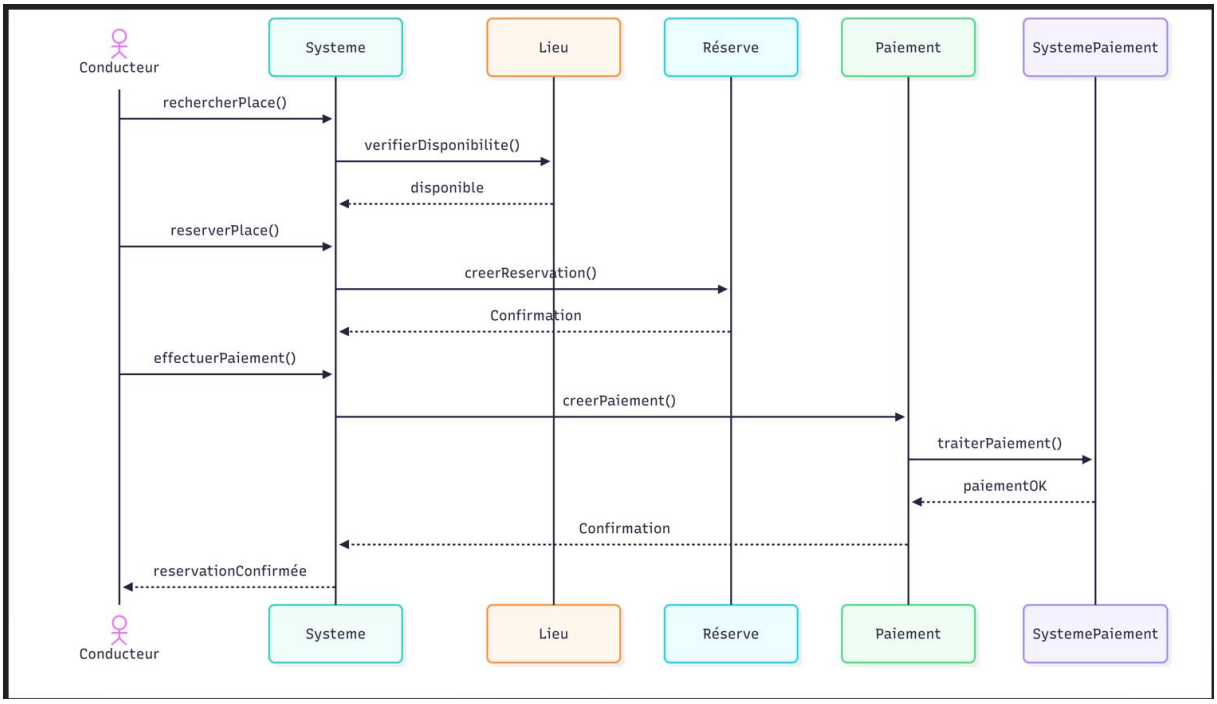



Figure : Diagramme de sequence

### IV.2.4 Diagramme d'activité

Le diagramme d'activité représente le flux de travail global du système.

Le processus se déroule comme suit :

- L'utilisateur commence par rechercher une place
- Le système vérifie la disponibilité

- Si la place est disponible, la réservation est effectuée
- Ensuite, l'utilisateur procède au paiement
- Si le paiement est validé, la réservation est confirmée
- Sinon, la réservation est annulée

Ce diagramme permet de visualiser les différentes étapes du processus ainsi que les décisions prises à chaque étape.

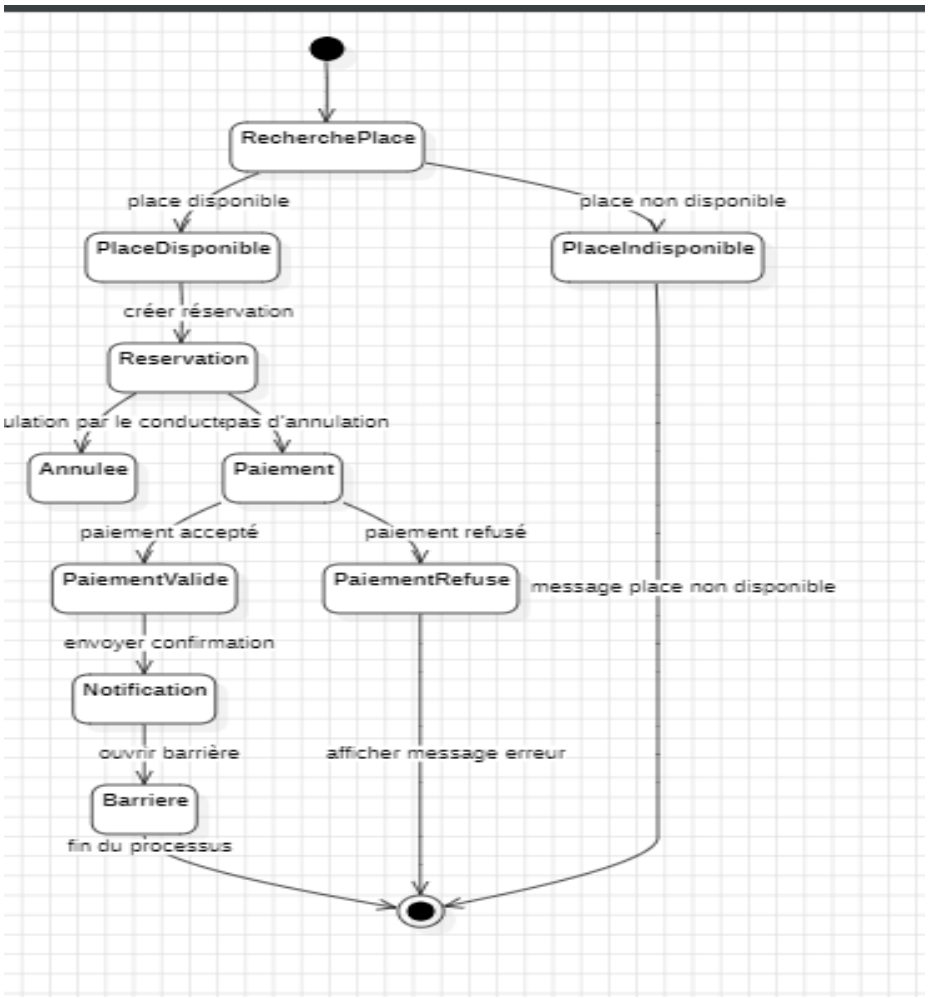



Figure : Diagramme d'activité

### IV.2.5 Diagramme de d'état

Le **diagramme d'état** est un diagramme UML qui permet de représenter **l'évolution d'un objet ou d'un composant du système** en fonction des événements qui se produisent. Il est particulièrement utile pour modéliser des entités qui changent d'état dans le temps, comme une **place de parking** ou une **réservation** dans notre système.

Ce diagramme est un diagramme d'états-transitions (UML) qui illustre le cycle de vie d'une ressource, probablement une place de parking ou un service de location de véhicule.

Voici une description détaillée des éléments du diagramme :

1. Les États (Rectangles arrondis)

Le système peut se trouver dans l'un des quatre états suivants :

* Disponible : L'état de repos où la ressource est prête à être utilisée.

* Reservee (Réservée) : La ressource a été bloquée par un utilisateur mais n'est pas encore occupée physiquement.

* Occupee (Occupée) : La ressource est actuellement utilisée (le véhicule est stationné).

* Indisponible : La ressource ne peut pas être utilisée en raison d'un problème technique.

2. Les Transitions (Flèches)

Les flèches décrivent les événements qui font passer le système d'un état à un autre :

| De l'état... | À l'état... | Événement déclencheur |

|---|---|---|

| Disponible | Reservee | Réservation créée |

| Reservee | Occupee | Véhicule stationne |

| Reservee | Disponible | Annulation de réservation |

| Occupee | Disponible | Véhicule quitte |

| Disponible | Indisponible | Maintenance ou panne |

| Indisponible | Disponible | Maintenance terminée |

3. Points de contrôle (Symboles spéciaux)

* Initial (à gauche, partiellement visible) : Le point de départ du processus (souvent vers l'état "Disponible").

* Fin (Cercle avec point noir) : Représente la fin du cycle de vie de la ressource dans le système (par exemple, si la place est définitivement supprimée). On remarque que tous les états principaux peuvent mener directement à l'état final.

Résumé du fonctionnement

Le cycle nominal est simple : une place disponible est réservée, puis devient occupée quand le véhicule arrive, et redeviendra disponible une fois que le véhicule part. Le diagramme prévoit aussi les cas d'exception comme l'annulation d'une réservation ou la mise hors service pour maintenance.

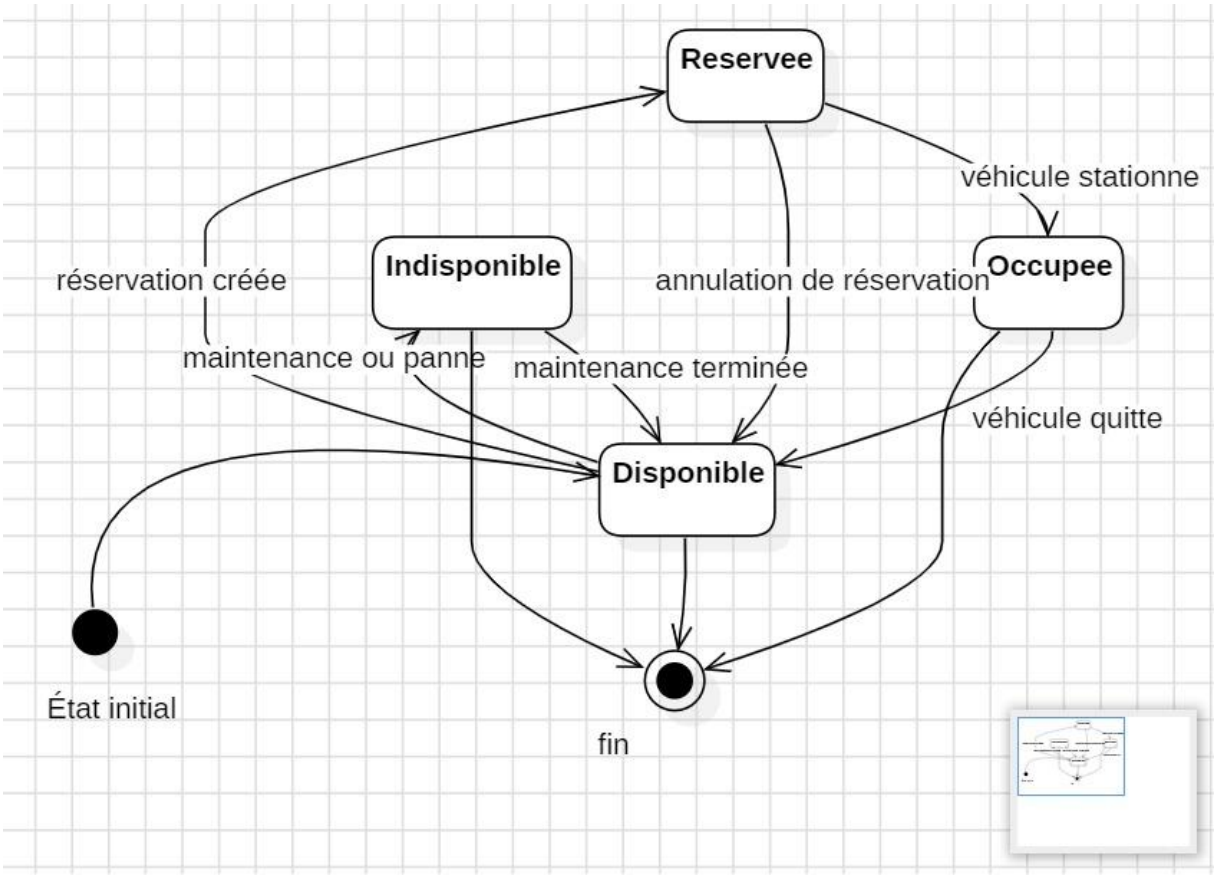



Figure : diagramme d'état

### IV.2.6 Diagramme de Collaboration

Le **diagramme de collaboration** est un diagramme UML qui décrit **les interactions entre les objets du système** et **la manière dont ils collaborent pour réaliser un scénario spécifique.**

Contrairement au **diagramme de séquence,** qui met l'accent sur **l'ordre temporel des messages,** le diagramme de collaboration **met l'accent sur la structure et les relations entre les objets** impliqués.

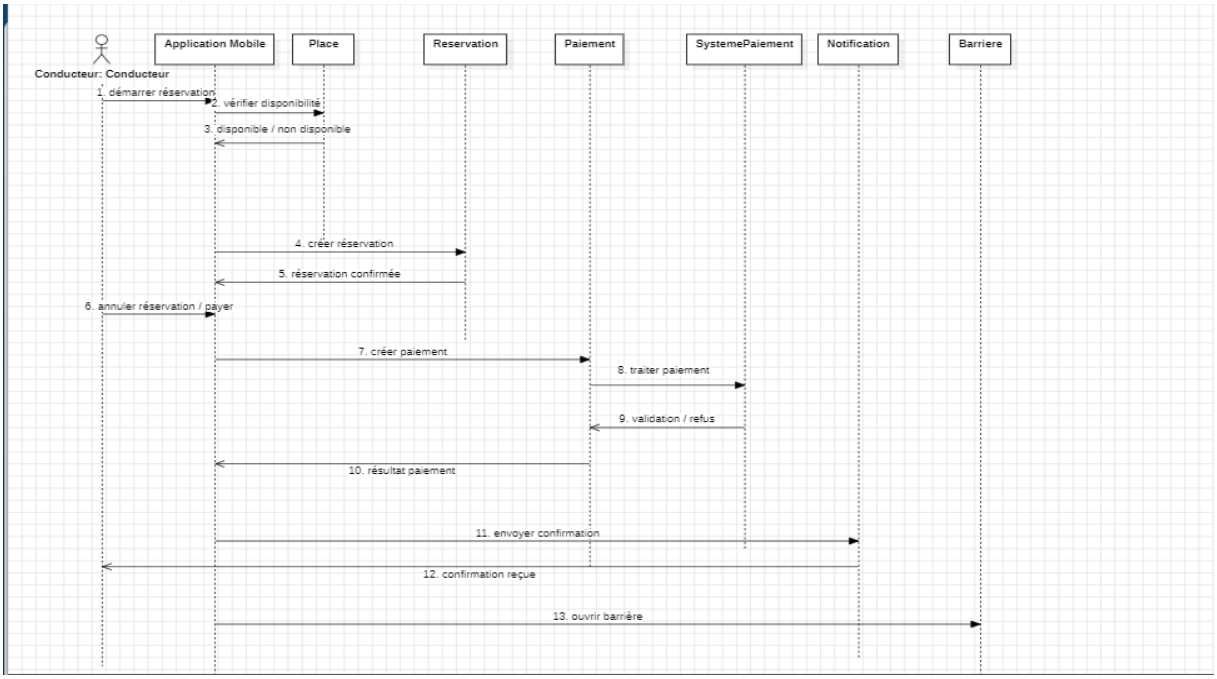



Figure : Diagramme de collaboration.

### IV.3 LA CONCEPTION DE BASE DES DONNEES

La conception de la base de données du système Moubarak Parking est une étape essentielle visant à assurer une gestion optimale, cohérente et sécurisée de toutes les informations liées au fonctionnement du parking intelligent. Dans ce cadre, le choix s'est porté sur une base de données relationnelle telle que MySQL, reconnue pour sa fiabilité, sa simplicité d'utilisation et sa capacité à gérer efficacement des relations entre différentes entités. La structure de la base repose sur plusieurs tables interconnectées, chacune représentant une entité clé du système. La table des utilisateurs contient les informations personnelles nécessaires à l'identification des clients et des administrateurs, telles que le nom, l'adresse email, le mot de passe et le rôle. La table des parkings regroupe les données relatives aux différents espaces de stationnement, notamment leur nom, leur localisation et leur capacité totale. Chaque parking est composé de plusieurs places, gérées dans une table dédiée qui permet de suivre en temps réel leur état (libre ou occupée).

Par ailleurs, la table des réservations joue un rôle central en assurant la liaison entre les utilisateurs et les places de parking. Elle enregistre les informations relatives à chaque réservation, comme les dates de début et de fin, ainsi que le statut (confirmée, annulée ou en attente). Cette organisation permet de gérer efficacement la disponibilité des places et d'éviter les conflits de réservation. En complément, une table de paiement est intégrée afin de suivre les transactions effectuées par les utilisateurs. Elle contient des informations telles que le montant payé, la date de paiement et le mode de règlement, garantissant ainsi une traçabilité financière complète du système.

Les relations entre ces différentes tables sont établies à l'aide de clés primaires et étrangères, assurant l'intégrité référentielle des données. Par exemple, chaque réservation est liée à un utilisateur et à une place spécifique, tandis que chaque paiement est associé à une réservation unique. Cette structuration permet non seulement d'optimiser les performances des requêtes, mais aussi de faciliter l'évolution future du système, comme l'ajout de nouvelles fonctionnalités. Enfin, des mesures de sécurité sont mises en place pour protéger les données sensibles, notamment le chiffrement des mots de passe, la gestion des droits d'accès selon les rôles (administrateur ou utilisateur) et la mise en œuvre de sauvegardes régulières. Ainsi, la base de données de Moubarik Parking constitue un socle robuste garantissant le bon fonctionnement et la fiabilité de l'ensemble du système.

# V . Réalisation et implémentation

## V .1 Environnement de développement

Dans le cadre de la réalisation de notre application de parking intelligent, nous avons mis en place un environnement de développement adapté permettant de concevoir, tester et déployer efficacement la solution.

Le développement a été réalisé sur un ordinateur personnel équipé d'un système d'exploitation Windows 11. Les outils suivants ont été utilisés :

- Visual Studio Code : éditeur de code principal
- Node.js : pour la gestion des dépendances
- Navigateur web (Microsoft ) : pour les tests
- WampServer : pour la base de données

Cet environnement nous a permis de travailler de manière structurée et organisée tout au long du projet.

## V .2 Technologies Utilisées

### V .2.1 React.js — Framework Frontend

React.js est une bibliothèque JavaScript utilisée pour développer l'interface utilisateur de l'application MOUBARIK Parking. Elle permet de créer des interfaces dynamiques et interactives à partir de composants réutilisables.

Dans ce projet, React est utilisé avec les hooks modernes tels que :

- **useState** : pour gérer les données de l'application
- **useEffect** : pour exécuter des actions automatiques
- **useContext** : pour gérer des informations globales comme l'authentification

Grâce à React, l'application est rapide, fluide et facile à maintenir.

**Résumé :** React.js permet de créer une interface utilisateur moderne, interactive et bien structurée.

### V .2.2 PHP — Backend et API REST

Le backend de l'application est développé avec PHP, un langage exécuté côté serveur. Il permet de traiter les données envoyées par le frontend et de gérer les fonctionnalités principales comme les réservations et les utilisateurs.

L'application utilise une API REST, ce qui permet au frontend React de communiquer avec le serveur à travers des requêtes HTTP.

La sécurité est assurée grâce à :

- l'authentification JWT
- l'utilisation de requêtes préparées pour éviter les attaques

**Résumé :** PHP permet de gérer la logique du système et assure la communication sécurisée entre le frontend et la base de données.

### V.2.3 MySQL — Base de Données

MySQL est utilisé pour stocker toutes les données de l'application. Il s'agit d'un système de gestion de base de données relationnelle.

Les données sont organisées en plusieurs tables liées entre elles, comme :

- utilisateurs
- parkings
- places
- réservations
- paiements

La base de données contient également des informations réelles du projet, notamment :

- plusieurs parkings
- des places disponibles
- une tarification définie
- des moyens de paiement locaux

**Résumé :** MySQL permet de stocker et organiser efficacement toutes les données de l'application.

### V.2.4 Bibliothèques complémentaires

Plusieurs bibliothèques ont été utilisées pour améliorer les fonctionnalités de l'application et offrir une meilleure expérience utilisateur.

- **React Router DOM** : gère la navigation entre les pages
- **Lucide React** : fournit des icônes modernes
- **jsPDF** : permet de générer des factures en PDF
- **QRCode.react** : permet de créer des QR codes pour les réservations
- **Supabase JS** : prépare une future évolution vers le cloud

Ces outils permettent d'ajouter des fonctionnalités avancées sans complexifier le développement.

**Résumé :** Les bibliothèques complémentaires enrichissent l'application avec des fonctionnalités modernes et améliorent l'expérience utilisateur.

### Conclusion de la section

Les technologies utilisées dans ce projet (React, PHP, MySQL et bibliothèques) permettent de créer une application complète, performante et évolutive, répondant aux besoins d'un système de parking intelligent.

## 5.3 Présentation des Fonctionnalités Principales

### V.3.1 Recherche de place disponible

La fonctionnalité de recherche permet au conducteur de connaître rapidement les places disponibles dans les parkings.

Elle est accessible depuis deux pages principales :

- **Dashboard (/dashboard)** : affiche en temps réel les statistiques globales (places libres, occupées et réservées), ainsi que les parkings proches avec leur taux d'occupation.
- **Plan (/plan)** : propose une vue détaillée des places par niveau (RDC, B1, B2), avec un code couleur :
    - Vert : disponible
    - Rouge : occupée
    - Orange : réservée

Les informations sont mises à jour automatiquement toutes les 5 secondes, garantissant une visualisation en temps réel.

**Résumé :** Cette fonctionnalité permet de visualiser rapidement les places disponibles grâce à un affichage clair et actualisé en temps réel.

### V.3.2 Réservation de place

La réservation se fait en trois étapes simples :

1. Choix du parking via une liste avec recherche
2. Sélection du niveau et de la place disponible
3. Confirmation de la réservation

Après validation :

- une réservation est créée
- un QR code est généré
- la place est temporairement réservée

Le QR code sert de preuve d'accès au parking. Si le conducteur ne se présente pas dans un délai de 15 minutes, la réservation est automatiquement annulée.

**Résumé :** Cette fonctionnalité permet de réserver facilement une place et garantit sa disponibilité pendant un temps limité.

### V.3.3 Navigation vers le parking

L'application permet au conducteur de se rendre facilement au parking choisi.

Chaque établissement possède des coordonnées GPS. Un bouton « Itinéraire » permet d'ouvrir directement la navigation via Google Maps.

Cela permet :

- de localiser rapidement le parking
- d'obtenir un itinéraire précis
- d'améliorer l'expérience utilisateur

**Résumé :** Cette fonctionnalité facilite l'accès au parking grâce à la navigation GPS intégrée.

### V.3.4 Paiement en ligne

Le paiement intervient à la fin de la session de stationnement.

Le montant est calculé automatiquement en fonction de la durée :

- 200 FDJ par 30 minutes

- maximum 3000 FDJ par jour

Deux modes de paiement sont disponibles :

- D-Money
- Espèces

Après paiement :

- la réservation est terminée
- la place est libérée
- une facture PDF est générée

**Résumé :** Cette fonctionnalité permet de payer facilement et automatiquement le stationnement avec génération de facture.

### V.3.5 Historique des réservations

L'utilisateur peut consulter toutes ses réservations dans une page dédiée.

Pour chaque réservation, il peut voir :

- la place
- le parking
- la durée
- le montant
- le statut

Des actions sont possibles :

- télécharger la facture
- afficher le QR code
- annuler une réservation
- laisser un avis

**Résumé :** Cette fonctionnalité permet de suivre et gérer toutes les réservations de manière simple et organisée.

### Conclusion de la section

L'ensemble de ces fonctionnalités permet de créer une application complète de gestion de parking, offrant une expérience utilisateur fluide, intuitive et efficace.

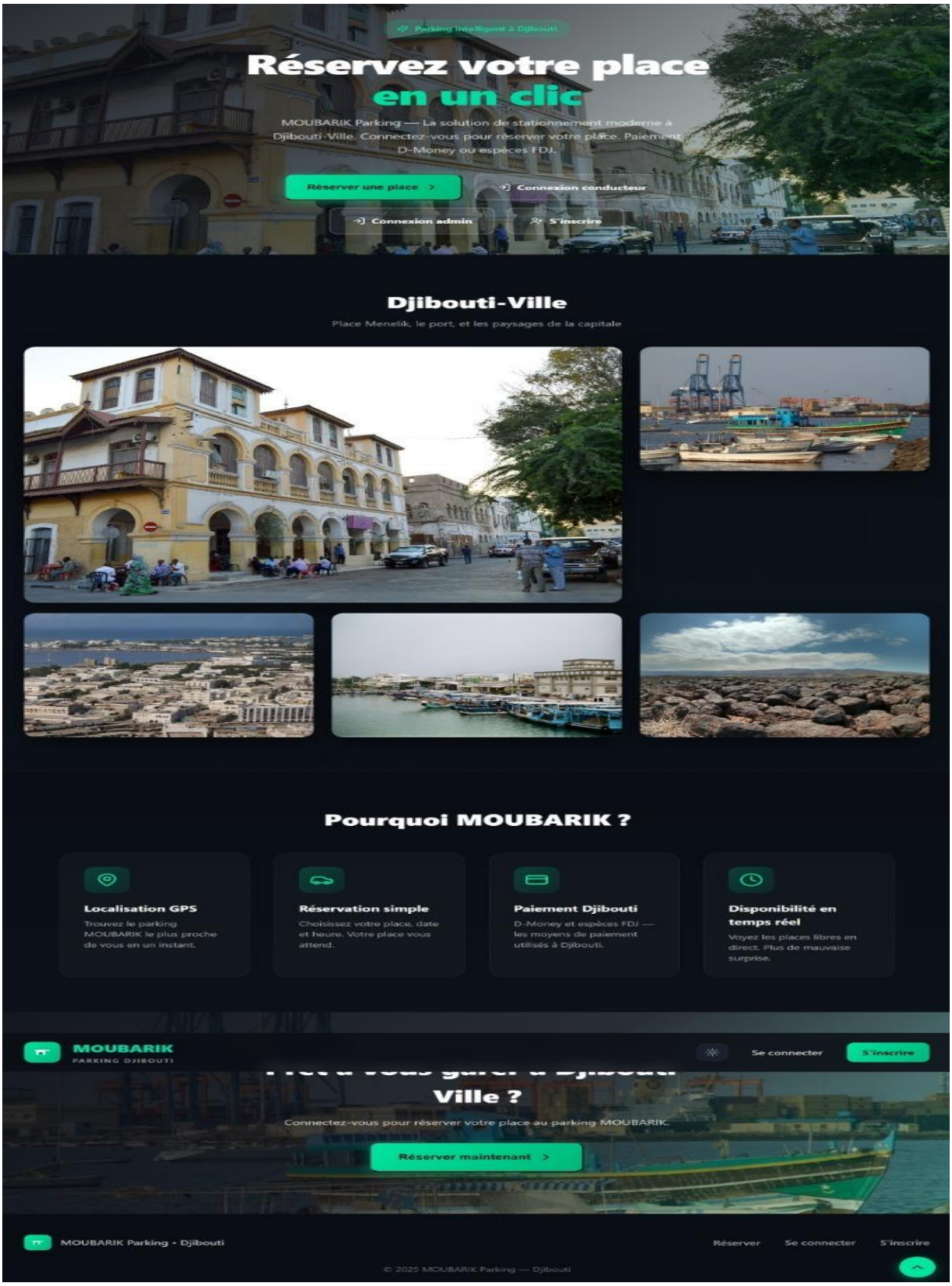
## V.4 Captures d’écran de l’application

### V.4.1 Interfaces Publiques

#### Page d’Accueil (Route : /)

La page d’accueil constitue le point d’entrée principal de l’application. Elle présente un **carrousel d’images** mettant en valeur des lieux emblématiques de Djibouti, comme la Place Menelik et le Port de Djibouti.
Cette interface a pour objectif de :

- Attirer l’utilisateur visuellement
- Présenter brièvement les services de l’application
- Donner une première impression professionnelle et moderne

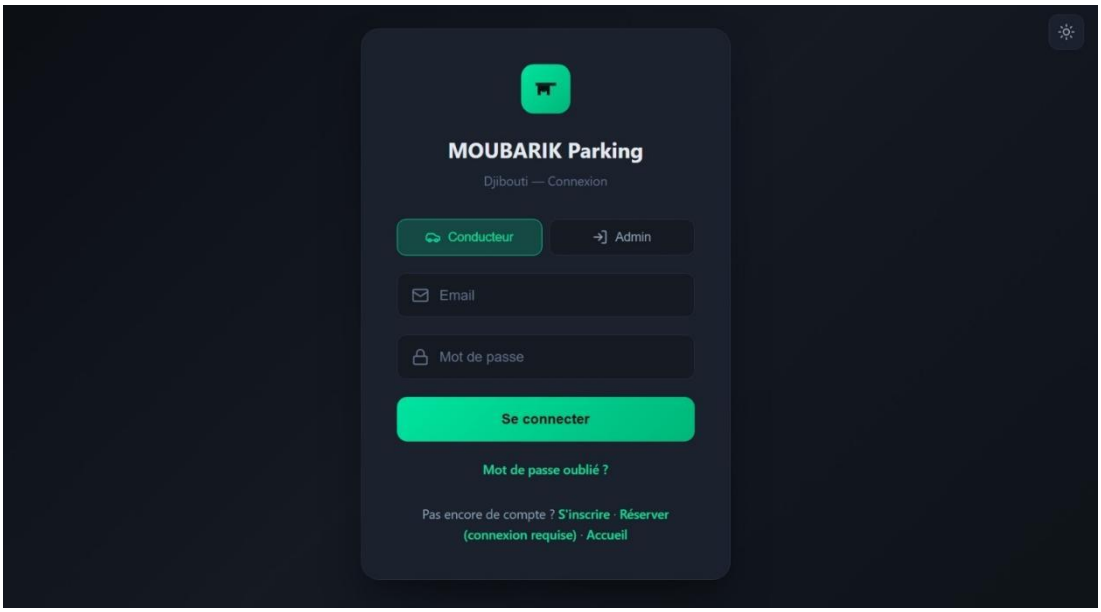

Parking intelligent à Djibouti
Réservez votre place
en un clic
MOUBARIK Parking — La solution de stationnement moderne à Djibouti-Ville. Connectez-vous pour réserver votre place. Paiement D-Money ou espèces FDJ.
Réserver une place
Connexion conducteur
Connexion admin
S'inscrire
Djibouti-Ville
Place Menelik, le port, et les paysages de la capitale
Pourquoi MOUBARIK ?
Localisation GPS
Trouvez le parking MOUBARIK le plus proche de vous en un instant.
Réservation simple
Choisissez votre place, date et heure. Votre place vous attend.
Paiement Djibouti
D-Money et espèces FDJ — les moyens de paiement utilisés à Djibouti.
Disponibilité en temps réel
Voyez les places libres en direct. Plus de mauvaise surprise.
MOUBARIK
PARKING DJIBOUTI
Se connecter
S'inscrire
Ville ?
Connectez-vous pour réserver votre place au parking MOUBARIK.
Réserver maintenant
MOUBARIK Parking - Djibouti
Réserver
Se connecter
S'inscrire
© 2025 MOUBARIK Parking — Djibouti

## Page de Connexion (Route : /connexion)

Cette page permet aux utilisateurs déjà inscrits d'accéder à leur compte.
Elle contient :

- Un champ email
- Un champ mot de passe
- Des liens vers **inscription** et **mot de passe oublié**

Son rôle est d'assurer une **authentification sécurisée** des utilisateurs.

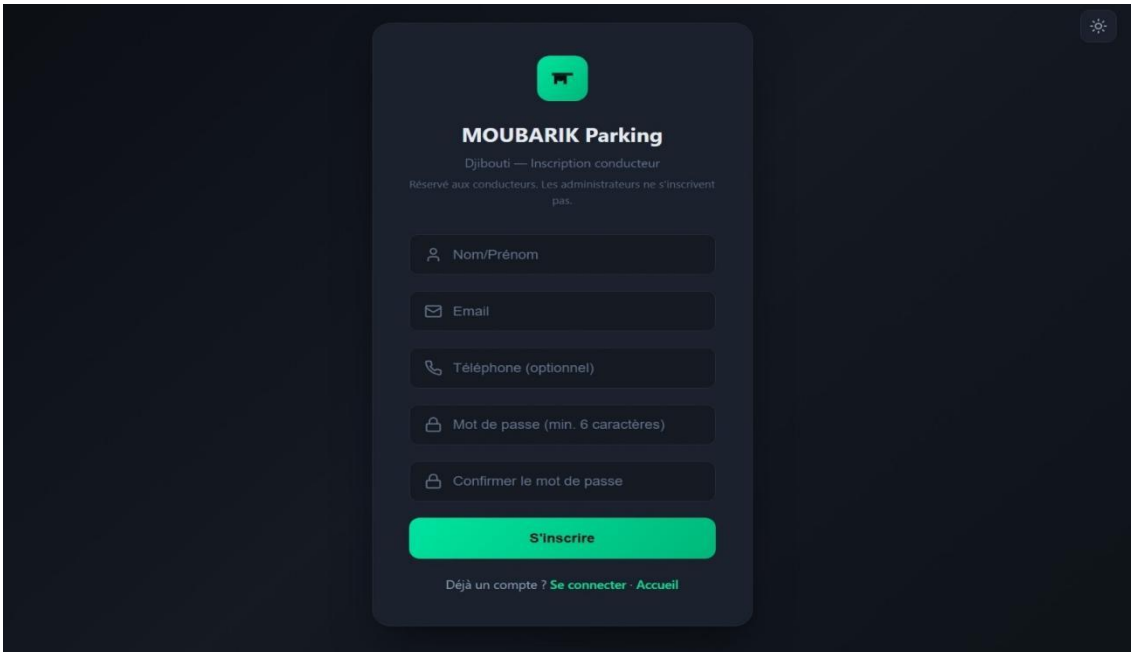



## Page d'Inscription (Route : /inscription)

Cette interface permet la création d'un nouveau compte utilisateur.
Elle inclut :

- Nom et prénom
- Email
- Numéro de téléphone
- Mot de passe avec validation

Elle garantit la **collecte structurée des informations utilisateur** avec des contrôles de validation pour éviter les erreurs.



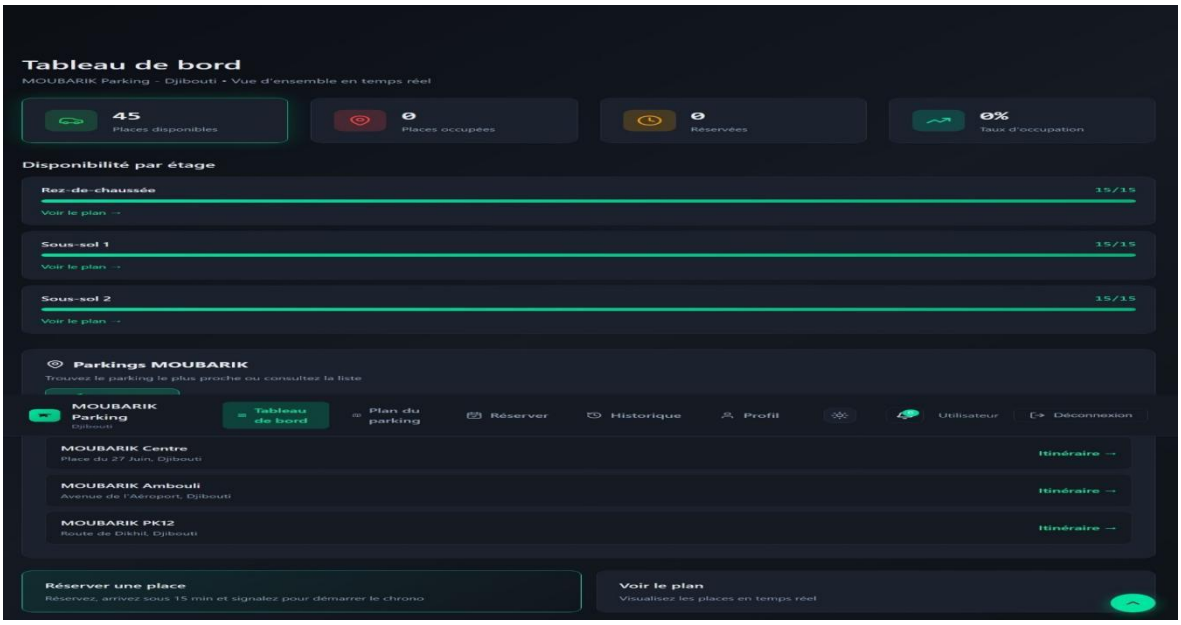

## V.4.2 Interface Conducteur

### Dashboard Conducteur (Route : /dashboard)

Le tableau de bord est l'interface principale après connexion.
Il affiche :

- Les statistiques globales du parking (places libres, occupées, réservées)
- Un chronomètre de session
- Les parkings proches via la géolocalisation

Cette interface permet une **vision rapide et en temps réel** de l'état du parking.

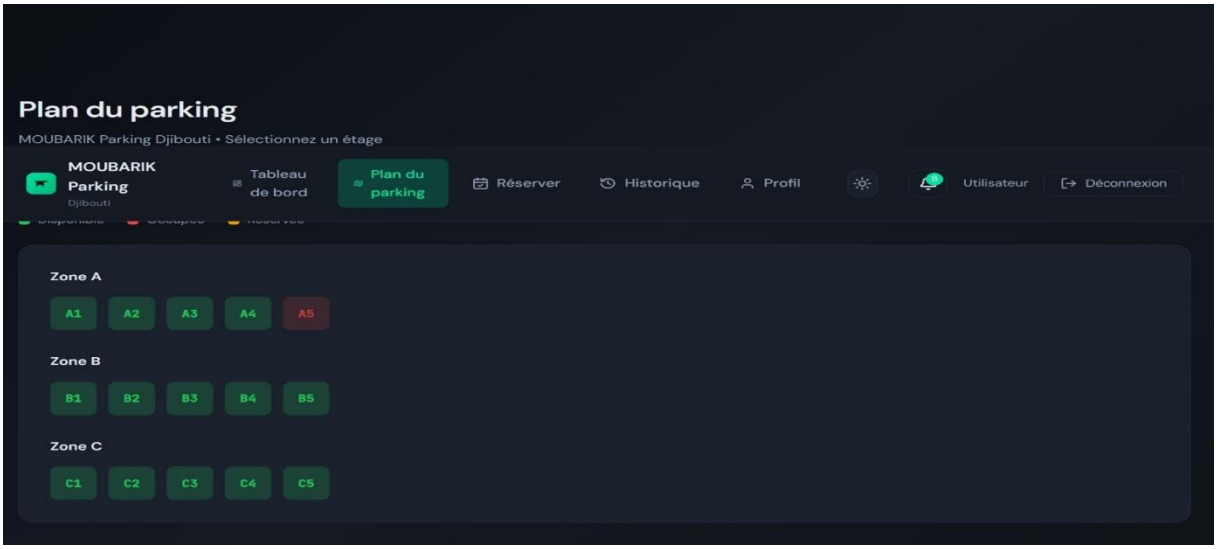


## Plan des Places (Route : /plan)

Cette interface affiche une **représentation visuelle du parking** sous forme de grille.
Les couleurs utilisées sont :

- Vert : place libre
- Rouge : place occupée
- Orange : place réservée

Elle permet à l'utilisateur de **visualiser rapidement la disponibilité des places**.

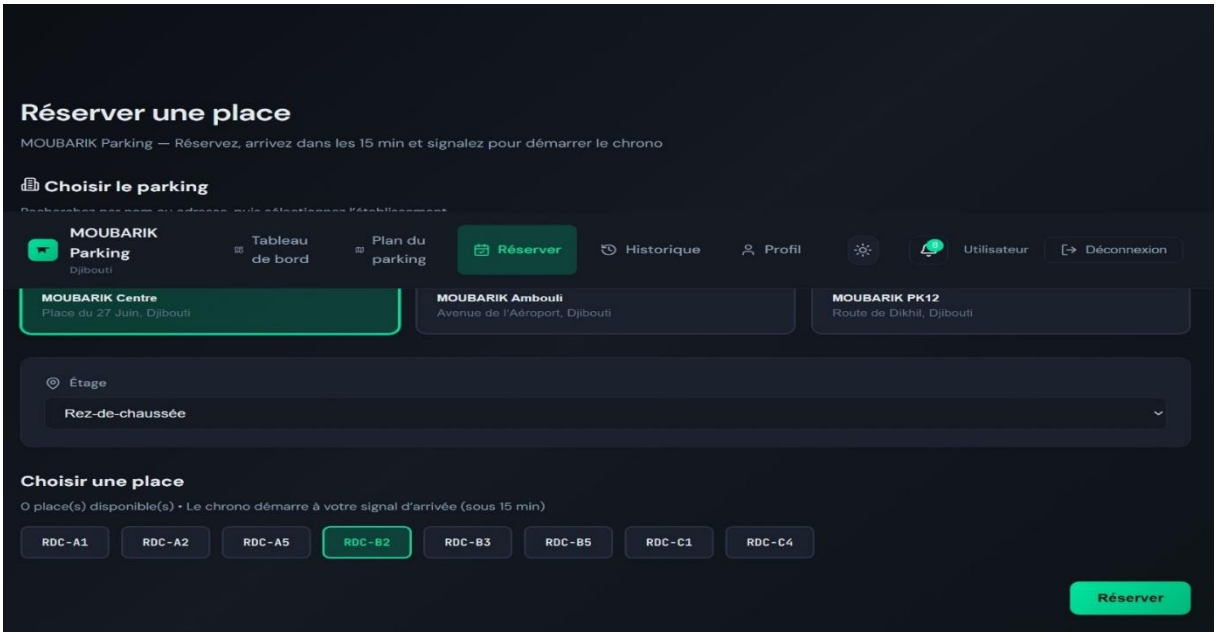



## Réservation (Route : /reservation)

Cette page permet à l'utilisateur de réserver une place de parking.
Elle affiche :

- Un niveau spécifique (ex : B1)
- Des places cliquables disponibles

Elle facilite la **sélection interactive et intuitive d'une place**.

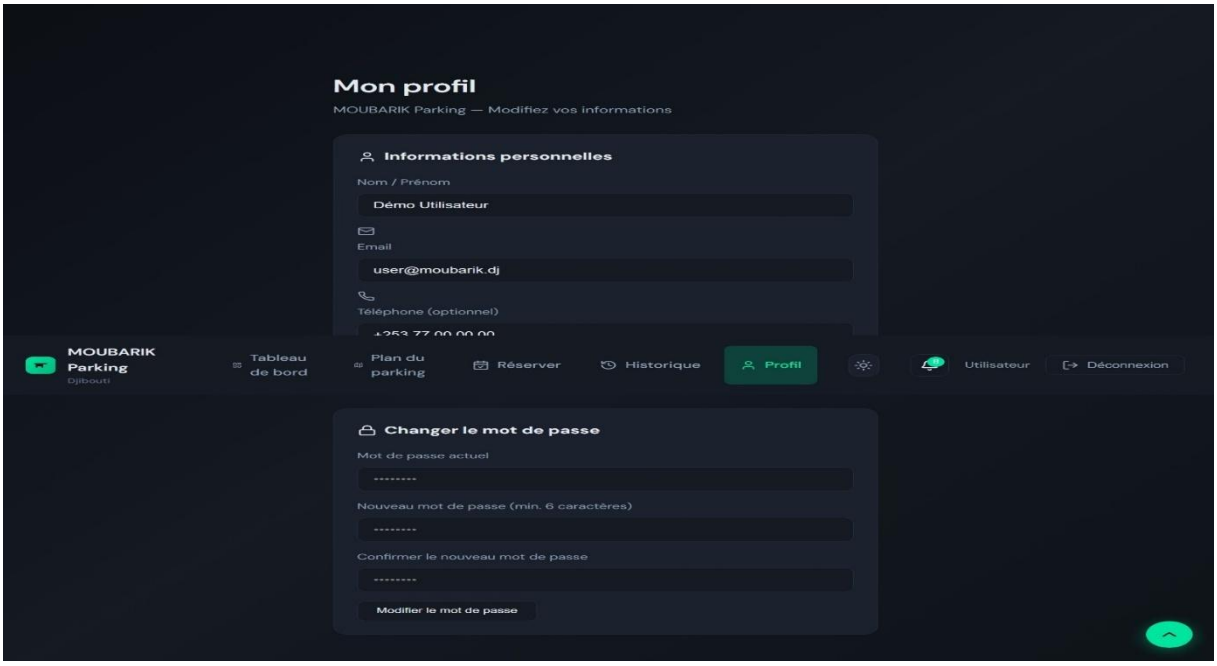



## Profil Conducteur (Route : /profil)

Cette interface regroupe les informations personnelles de l'utilisateur.
Elle permet :

- La modification des informations
- Le changement de mot de passe
- La gestion des notifications

Elle assure une **gestion complète du compte utilisateur**.



## V.4.3 Interface Administrateur

### Tableau de Bord Administrateur (Route : /admin)

Cette interface fournit une vue globale du système.
Elle affiche :

- Nombre total de places
- Taux d'occupation
- Réservations du jour
- Recettes générées

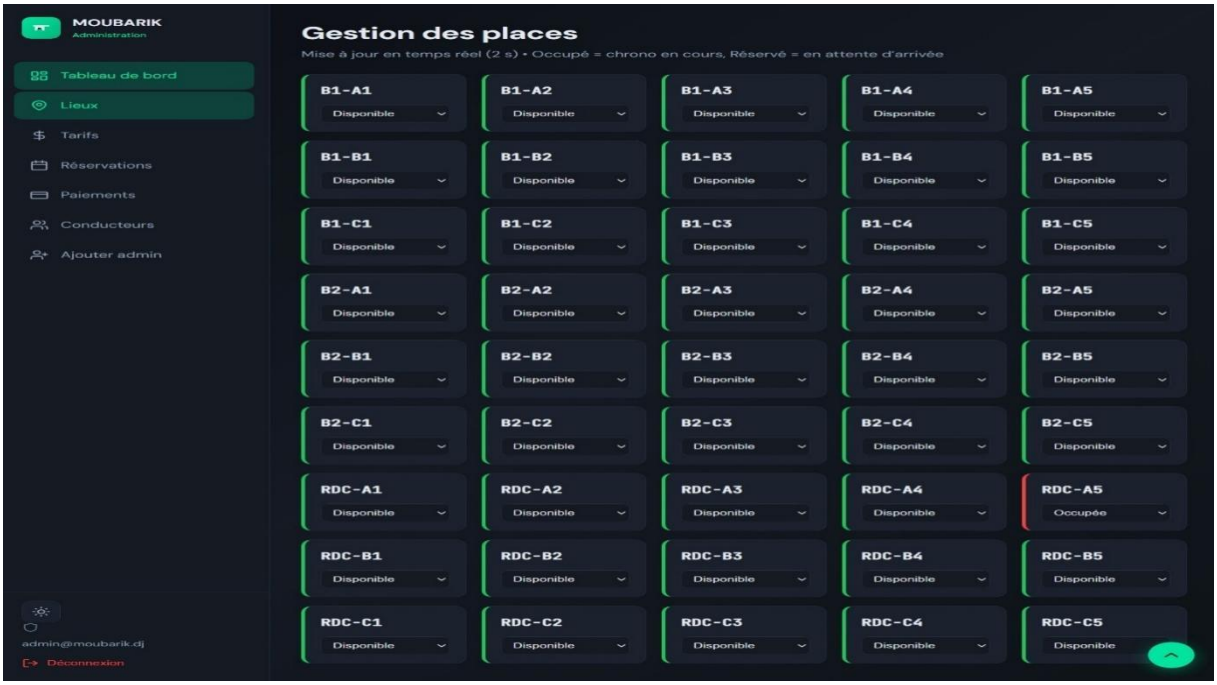
Elle permet un **suivi global de l'activité du parking**.

### Gestion des Places (Route : /admin/places)

Cette page permet de gérer les 45 places disponibles.
L'administrateur peut :

- Consulter l'état des places
- Modifier leur statut

Elle assure une **gestion dynamique et centralisée du parking**.




## Gestion des Tarifs (Route : /admin/tarifs)

Cette interface permet de définir les prix du parking.
Exemple :

- 200 FDJ / 30 minutes
- 3000 FDJ / jour

Elle permet de :

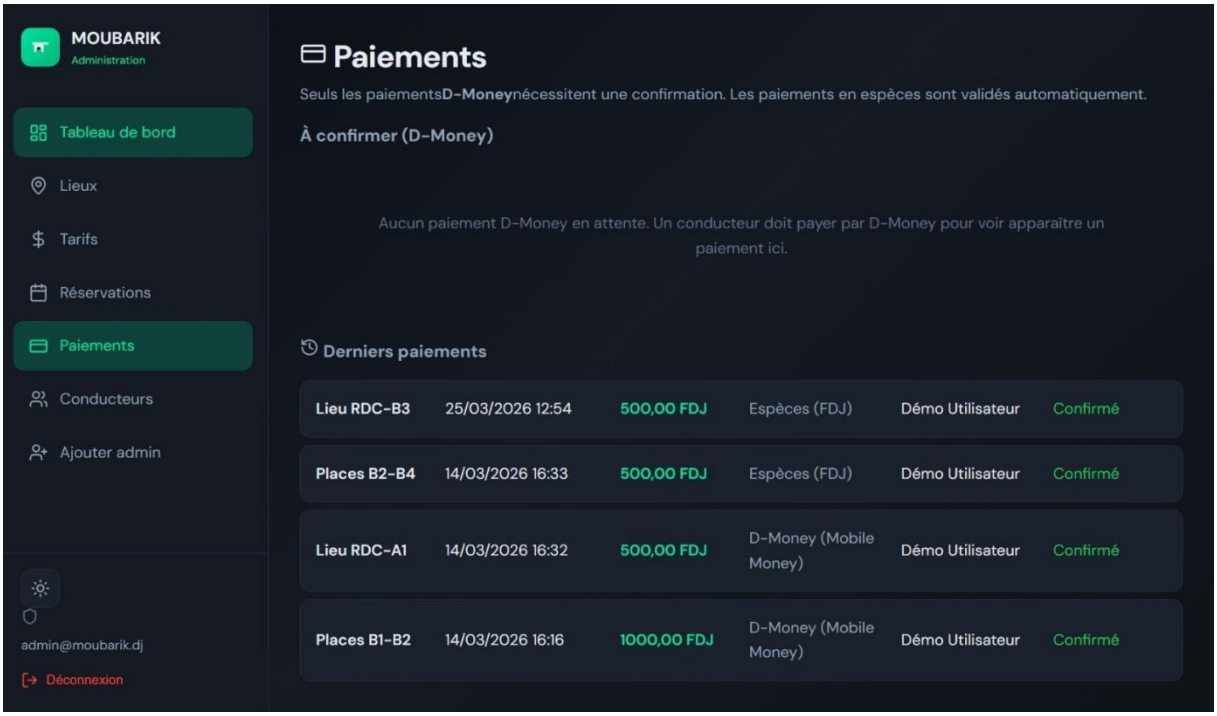
- Modifier les tarifs en temps réel
- Adapter la politique de prix

## Gestion des Paiements (Route : /admin/paiements)

Cette page affiche toutes les transactions effectuées.
Elle permet :

- Le filtrage par méthode (D-Money, espèces)
- Le suivi du statut des paiements

Elle garantit une **traçabilité financière complète**.



### V.4.4 Thème Clair et Thème Sombre

L'application propose deux modes d'affichage : le **thème clair** et le **thème sombre**.
Par défaut, l'interface est configurée en **mode sombre**, offrant un confort visuel optimal, notamment en environnement peu éclairé et en réduisant la fatigue oculaire.

L'utilisateur a toutefois la possibilité de basculer vers le **thème clair** selon ses préférences. Le choix du thème est sauvegardé automatiquement via **localStorage**, garantissant une expérience personnalisée et persistante lors des prochaines utilisations.

### Conclusion de la section

La section des captures d'écran présente les différentes interfaces de l'application, regroupées en interfaces publiques, conducteur et administrateur.

L'application offre une expérience complète avec une navigation intuitive, la gestion des réservations, ainsi qu'un système d'administration performant pour la gestion des places, des tarifs et des paiements.

Elle intègre également des fonctionnalités modernes comme la géolocalisation et le mode clair/sombre, garantissant une utilisation fluide, ergonomique et adaptée aux besoins des utilisateurs.

# VI . Tests ,Déploiement et Maintenance

## VI .1 Stratégie de Test

Pour vérifier que l'application fonctionne correctement, plusieurs types de tests ont été réalisés :

- les **tests unitaires** (vérifient chaque petite partie du système),
- les **tests fonctionnels** (vérifient que les fonctionnalités marchent du début à la fin),
- les **tests d'interface** (vérifient l'affichage et l'expérience utilisateur).

Comme il s'agit d'un projet académique, les tests ont été faits **manuellement**, sans outil automatique.

Chaque fonctionnalité a été testée selon trois points :

- fonctionnement normal
- gestion des erreurs
- cas limites

Les résultats ont été enregistrés dans un tableau de suivi global des tests présent en annexe A **(voir annexe).**

## VI.2 Tests Fonctionnels

Les tests fonctionnels permettent de vérifier que chaque module de l'application marche correctement.

### Authentification

- Vérifie la connexion des utilisateurs (admin et conducteur)
- Teste les erreurs comme mauvais mot de passe ou email inexistant
- Vérifie la sécurité des routes (accès sans connexion)

---

### Réservation

- Permet de réserver une place disponible
- Gère l'expiration automatique
- Empêche la double réservation
- Permet l'annulation selon des règles de temps

---

### Paiement

- Vérifie le calcul des montants
- Gère les paiements (D-Money et espèces)
- Assure la détection des erreurs (double paiement)

---

### Administration

- Permet de gérer les places, les tarifs et les utilisateurs
- Donne accès aux statistiques globales
- Assure la création et la gestion des comptes administrateurs

## VI.3 Tests d'Interface

### Compatibilité et réactivité

L'application a été testée sur différents navigateurs (Chrome, Edge, Firefox) et appareils (ordinateur et mobile).

Résultat :

- L'application fonctionne correctement sur la majorité des plateformes
- Un problème partiel a été détecté sur Safari mobile (téléchargement PDF)

---

### Thème clair / sombre

- Le thème peut être changé facilement
- Le choix est conservé automatiquement
- L'affichage reste lisible dans les deux modes

---

### Performance

Les performances sont bonnes :

- chargement rapide
- temps de réponse faible
- application optimisée

## VI.4 Résultats et Corrections

### Bilan des tests

- La majorité des tests ont réussi
- Un seul problème partiel a été détecté
- Tous les autres tests sont validés

---

### Corrections apportées

- Correction du calcul des prix (arrondi correct)
- Amélioration de l'actualisation des places en temps réel

### Limites et recommandations

- Problème de téléchargement PDF sur iOS
- Absence de tests automatisés
- Paiement D-Money simulé (non réel)
- Nécessité d'ajouter HTTPS en production

## VI.5 Déploiement

### VI.5.1 Déploiement en environnement de développement local

Le déploiement en environnement local permet de tester l'application sur la machine de développement avant sa mise en ligne.

Les étapes réalisées sont les suivantes :

1. Installation des dépendances du projet :

```
npm install
```

2. Démarrage du serveur web local (WampServer) afin d'exécuter le backend PHP et la base de données MySQL.
3. Importation ou initialisation de la base de données via le script d'installation :

```
http://localhost:8000/install.php
```

4. Lancement des deux serveurs :
   - Frontend React (Vite) :

```
npm run dev
```

   - Backend PHP :

```
npm run dev:api
```

L'application est alors accessible localement à l'adresse :

**http://localhost:5173**

Ce mode de déploiement permet de vérifier le bon fonctionnement de toutes les fonctionnalités avant la mise en production.

### VI.5.2 Déploiement en environnement de production

Une fois l'ensemble des tests nécessaires effectués et validés, garantissant le bon fonctionnement et la stabilité du système, l'application a été déployée en environnement de production.

Le déploiement a été réalisé sur la plateforme d'hébergement gratuite **InfinityFree**, permettant de rendre le système accessible en ligne sans frais. Cette solution a été choisie dans le cadre de ce projet académique afin de faciliter la mise en ligne rapide de l'application tout en respectant les contraintes de ressources.

Le processus de déploiement a consisté à transférer les fichiers du projet vers le serveur d'hébergement, à configurer la connexion à la base de données distante, puis à vérifier le bon fonctionnement de l'ensemble des fonctionnalités en environnement réel.

Ce déploiement intervient après la phase de tests afin de s'assurer que le système est stable, sécurisé et conforme aux exigences fonctionnelles définies. Il permet également de valider le comportement de l'application dans des conditions proches d'une utilisation réelle.

Ainsi, cette étape marque la mise en service effective du système, le rendant accessible aux utilisateurs finaux via Internet, tout en garantissant un fonctionnement conforme aux attentes du projet.

**Resume :**

Le déploiement du système a permis de rendre l'application accessible et fonctionnelle, aussi bien en environnement local qu'en ligne.

Cette approche garantit une validation progressive du système et démontre la capacité de l'application à fonctionner dans un environnement réel, tout en restant adaptée à un cadre académique.

Conclusion de la section

Le processus de tests et de déploiement a permis de valider le bon fonctionnement de l'application MOUBARIK Parking.

Les différentes phases de test ont confirmé la conformité du système aux besoins définis, tandis que le déploiement a assuré sa mise à disposition en ligne, renforçant ainsi son caractère opérationnel et fonctionnel.

## VI.6 Maintenance du logiciel

La maintenance du logiciel constitue une phase essentielle du cycle de vie d'un système informatique, intervenant après son déploiement en environnement réel. Elle a pour objectif d'assurer le bon fonctionnement, la fiabilité et l'évolution continue de l'application, en tenant compte des besoins des utilisateurs et des contraintes techniques.

Après la mise en production, le logiciel peut présenter des anomalies ou nécessiter des améliorations afin de s'adapter à de nouvelles exigences. Ainsi, la maintenance permet non seulement de corriger les erreurs, mais également d'optimiser les performances et d'intégrer de nouvelles fonctionnalités.

On distingue généralement trois types principaux de maintenance :

### VI.6.1 Maintenance corrective

La maintenance corrective consiste à identifier et corriger les anomalies (bugs) détectées après la mise en service du logiciel. Ces erreurs peuvent affecter le bon fonctionnement du système ou l'expérience utilisateur.

Dans le cadre de notre application de parking intelligent, cela peut concerner, par exemple, la correction d'un dysfonctionnement lors de la réservation ou un problème d'affichage des places disponibles.

**Résumé :** La maintenance corrective vise à corriger les erreurs afin d'assurer le bon fonctionnement du système.

### VI.6.2 Maintenance évolutive

La maintenance évolutive a pour objectif d'améliorer le logiciel en ajoutant de nouvelles fonctionnalités ou en optimisant celles déjà existantes. Elle permet d'adapter l'application aux nouveaux besoins des utilisateurs.

Par exemple, l'ajout d'un nouveau mode de paiement ou l'intégration d'un système de notification constitue une évolution du système.

**Résumé :** La maintenance évolutive permet d'enrichir et d'améliorer les fonctionnalités du logiciel.

### VI.6.3 Maintenance adaptative

La maintenance adaptative consiste à modifier le logiciel afin qu'il reste compatible avec son environnement technique, notamment lors de changements de système d'exploitation, de navigateur ou de matériel.

Dans notre cas, cela peut inclure l'adaptation de l'application pour fonctionner sur différents navigateurs ou appareils mobiles.

**Résumé :** La maintenance adaptative permet d'assurer la compatibilité du logiciel avec son environnement.

**Conclusion de la section**

En conclusion, la maintenance du logiciel est une activité indispensable pour garantir sa pérennité, sa performance et sa capacité d'évolution. Elle permet d'assurer une meilleure qualité de service et de répondre efficacement aux attentes des utilisateurs dans le temps.

# CONCLUSION GENERALE

Au terme de ce travail, nous pouvons affirmer que le projet MOUBARIK Parking a atteint les objectifs fixés au départ. Cette application web de gestion de parking intelligent répond concrètement à un problème réel observé dans la ville de Djibouti : la difficulté à trouver et à réserver une place de stationnement de manière rapide, simple et fiable.

## Bilan du projet

Ce projet nous a permis de concevoir et de développer une solution complète, allant de l'analyse des besoins jusqu'au déploiement en ligne. Toutes les fonctionnalités principales ont été réalisées avec succès :

- La recherche et la visualisation en temps réel des places disponibles, grâce à un affichage par code couleur mis à jour automatiquement toutes les 5 secondes.
- La réservation en ligne avec génération d'un QR code unique servant de preuve d'accès au parking.
- La navigation GPS intégrée permettant au conducteur de rejoindre facilement le parking sélectionné.
- Le paiement automatique du stationnement, avec calcul du montant selon la durée et génération d'une facture PDF.
- L'historique complet des réservations, permettant à chaque utilisateur de suivre et gérer ses sessions passées.
- Un tableau de bord administrateur offrant une vision globale du système : occupation, recettes, gestion des places et des tarifs.

Sur le plan technique, l'architecture React / PHP / MySQL s'est révélée solide et adaptée au contexte du projet. L'utilisation de bibliothèques modernes telles que jsPDF, QRCode.react et React Router DOM a permis d'enrichir l'application sans alourdir le développement. La phase de tests a confirmé la stabilité du système, avec un seul problème partiel identifié sur Safari iOS concernant le téléchargement de PDF.

## Difficultés rencontrées

La réalisation de ce projet n'a pas été sans obstacles. Plusieurs difficultés ont été rencontrées tout au long du développement :

- La gestion du temps réel pour la mise à jour automatique des places a nécessité une réflexion approfondie sur le mécanisme de polling et la fréquence des appels à l'API.
- La sécurisation des routes et la mise en place de l'authentification JWT entre le frontend React et le backend PHP ont demandé une attention particulière pour éviter toute faille.
- L'intégration du paiement D-Money n'a pas pu être réalisée en mode réel faute d'accès à une API officielle ; elle reste simulée dans la version actuelle.
- La compatibilité avec Safari iOS a posé un problème spécifique lors du téléchargement de factures PDF, une contrainte propre à ce navigateur mobile.
- La gestion des cas limites dans la logique de réservation, notamment l'expiration automatique après 15 minutes et la prévention de la double réservation, a exigé une implémentation soignée côté backend.

## Perspectives et améliorations futures

Bien que l'application soit fonctionnelle et déployée, plusieurs pistes d'amélioration ont été identifiées pour faire évoluer le projet vers une solution encore plus complète et professionnelle :

- Intégration d'un paiement en ligne réel via l'API officielle D-Money ou d'autres solutions de paiement mobile disponibles à Djibouti.
- Migration vers une architecture cloud avec Supabase, déjà préparée dans le code, afin d'assurer une meilleure scalabilité et une disponibilité accrue.
- Développement d'une application mobile native (React Native) pour offrir une expérience encore plus fluide sur smartphone.
- Ajout d'un système de notification en temps réel (SMS ou push) pour alerter le conducteur avant l'expiration de sa réservation.
- Mise en place de tests automatisés (Jest, Cypress) pour fiabiliser les futures évolutions du système.
- Ajout de la reconnaissance de plaques d'immatriculation à l'entrée du parking pour automatiser l'accès et renforcer la sécurité.

En conclusion, ce projet constitue une réponse concrète et innovante aux besoins de modernisation de la gestion du stationnement à Djibouti. Il démontre la faisabilité technique d'une telle solution dans un contexte académique, tout en posant des bases solides pour une éventuelle mise en production à grande échelle.

# Bibliographie / Webographie


**Documentation technique officielle**

[1] Meta Platforms, Inc. — *React.js — Documentation officielle*. https://react.dev (consulté en 2024-2025)

[2] The PHP Group — *PHP Manual — Documentation officielle*. https://www.php.net/manual/fr (consulté en 2024-2025)

[4] WampServer — *Documentation officielle WampServer*. https://www.wampserver.com (consulté en 2024)

[5] Vite.js — *Vite — Next Generation Frontend Tooling*. https://vitejs.dev (consulté en 2024-2025)

**Bibliothèques et outils utilisés**

[6] npm — *react-router-dom — Declarative routing for React*. https://www.npmjs.com/package/react-router-dom (consulté en 2024-2025)

[7] Lucide — *Lucide React — Icon library for React applications*. https://lucide.dev/guide/packages/lucide-react (consulté en 2024-2025)

[8] npm — *jsPDF — PDF Document creation from JavaScript*. https://www.npmjs.com/package/jspdf (consulté en 2024-2025)

[9] npm — *qrcode.react — A React component to generate QR codes*. https://www.npmjs.com/package/qrcode.react (consulté en 2024-2025)

[10] Supabase — *Documentation officielle Supabase JS*. https://supabase.com/docs/reference/javascript/v1 (consulté en 2024-2025)

[11] Auth0 / JWT — *JSON Web Tokens — Introduction*. https://jwt.io (consulté en 2024)

**Méthodologie et gestion de projet**

[12] Schwaber, K. & Sutherland, J. — *The Scrum Guide*. https://scrumguides.org (consulté en 2024)

**Modélisation UML**

[14] Object Management Group — *UML 2.5 Specification*. https://www.omg.org/spec/UML (consulté en 2024)

# ANNEXES

## Annexe A — Tableau de suivi global des tests

| Module | Cas de test | Résultat attendu | Résultat obtenu | Statut |
|---|---|---|---|---|
| **Authentification** | Connexion admin | Redirection /admin + JWT | OK | ✓ |
| **Authentification** | Connexion conducteur | Redirection /dashboard | OK | ✓ |
| **Authentification** | Mot de passe incorrect | Message d'erreur | OK | ✓ |
| **Authentification** | Email inexistant | Compte introuvable | OK | ✓ |
| **Authentification** | Accès sans token | Redirection login | OK | ✓ |
| **Authentification** | Accès admin interdit | Redirection dashboard | OK | ✓ |
| **Réservation** | Réservation normale | QR + statut pending | OK | ✓ |
| **Réservation** | Expiration automatique | Place disponible | OK | ✓ |
| **Réservation** | Signalement arrivée | Place occupée | OK | ✓ |
| **Réservation** | Double réservation | Place indisponible | OK | ✓ |
| **Réservation** | Annulation valide | Place libérée | OK | ✓ |
| **Réservation** | Annulation tardive | Impossible d'annuler | OK | ✓ |
| **Paiement** | 30 minutes | 200 FDJ | OK | ✓ |
| **Paiement** | 1h15 | 750 FDJ | OK | ✓ |
| **Paiement** | Plafond journalier | 3000 FDJ max | OK | ✓ |
| **Paiement** | Paiement D-Money | Facture générée | OK | ✓ |
| **Paiement** | Paiement espèces | Paiement validé | OK | ✓ |
| **Paiement** | Double paiement | Erreur détectée | OK | ✓ |
| **Administration** | Dashboard admin | Données affichées | OK | ✓ |
| **Administration** | Modification place | Statut modifié | OK | ✓ |
| **Administration** | Modification tarif | Tarif appliqué | OK | ✓ |
| **Administration** | Liste conducteurs | Liste complète | OK | ✓ |
| **Administration** | Création admin | Compte créé | OK | ✓ |
| **Interface** | Chrome | Fonctionnel | OK | ✓ |

| | | | | |
|---|---|---|---|---|
| **Interface** | Edge | Fonctionnel | OK | ✓ |

## Annexe B — Script SQL de création de la base de données

Le script ci-dessous constitue le schéma complet de la base de données de l'application MOUBARIK Parking. Il crée toutes les tables, définit les relations entre elles et insère les données initiales nécessaires au fonctionnement du système. Ce script est à exécuter dans phpMyAdmin (http://localhost/phpmyadmin) via WampServer.

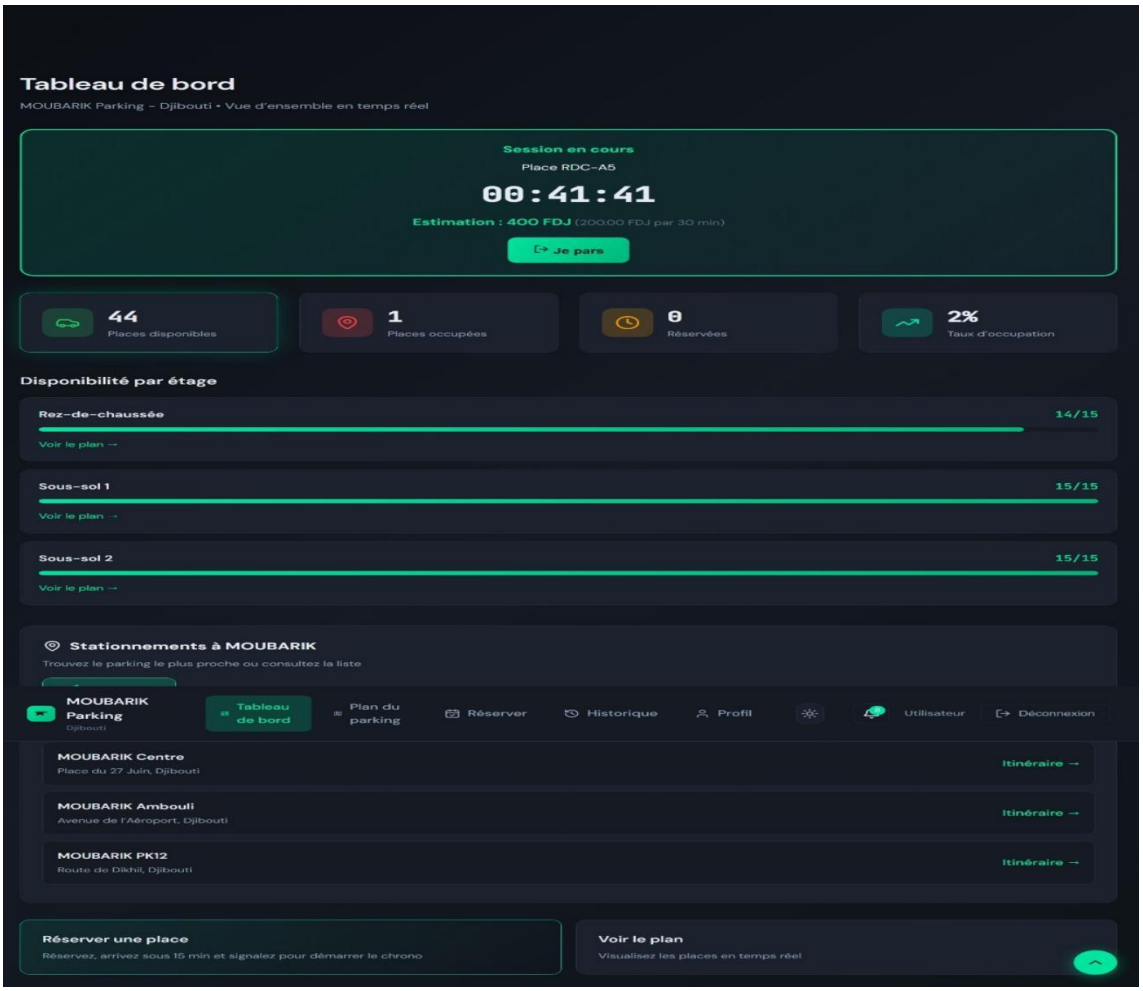
```
-- MOUBARIK Parking Djibouti - Schéma MySQL pour WAMP
-- Exécutez ce script dans phpMyAdmin (http://localhost/phpmyadmin)


CREATE DATABASE IF NOT EXISTS moubarik_parking;

USE moubarik_parking;


-- SystemePaiement
CREATE TABLE IF NOT EXISTS SystemePaiement (

  idSysteme INT AUTO_INCREMENT PRIMARY KEY,

  typeSysteme VARCHAR(50) NOT NULL

);


-- Tarif
CREATE TABLE IF NOT EXISTS Tarif (

  idTarif INT AUTO_INCREMENT PRIMARY KEY,

  description VARCHAR(255) NOT NULL,

  montantHoraire DECIMAL(10,2) NOT NULL DEFAULT 0,

  montantJournalier DECIMAL(10,2) NOT NULL DEFAULT 0

);


-- Administrateur
CREATE TABLE IF NOT EXISTS Administrateur (

  idAdmin INT AUTO_INCREMENT PRIMARY KEY,

  nom VARCHAR(100) NOT NULL,

  email VARCHAR(150) UNIQUE NOT NULL,

  motDePasse VARCHAR(255) NOT NULL,

  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);
```

```sql
-- Conducteur
CREATE TABLE IF NOT EXISTS Conducteur (
  idConducteur INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  telephone VARCHAR(20),
  motDePasse VARCHAR(255) NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Place
CREATE TABLE IF NOT EXISTS Place (
  idPlace INT AUTO_INCREMENT PRIMARY KEY,
  numero VARCHAR(20) UNIQUE NOT NULL,
  localisation VARCHAR(255) NOT NULL,
  statut ENUM('available','occupied','reserved') DEFAULT 'available',
  idTarif INT,
  FOREIGN KEY (idTarif) REFERENCES Tarif(idTarif)
);

-- Capteur
CREATE TABLE IF NOT EXISTS Capteur (
  idCapteur INT AUTO_INCREMENT PRIMARY KEY,
  idPlace INT UNIQUE NOT NULL,
  localisation VARCHAR(255),
  etat ENUM('empty','occupied') DEFAULT 'empty',
  derniereLecture TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (idPlace) REFERENCES Place(idPlace) ON DELETE CASCADE
);

-- Reservation
CREATE TABLE IF NOT EXISTS Reservation (
  idReservation INT AUTO_INCREMENT PRIMARY KEY,
  idConducteur INT,
  idPlace INT NOT NULL,
  idTarif INT,
  dateDebut DATETIME NOT NULL,
```

```
  dateFin DATETIME,
  montant DECIMAL(10,2) DEFAULT 0,
  statut ENUM('pending','confirmed','cancelled') DEFAULT 'confirmed',
  FOREIGN KEY (idConducteur) REFERENCES Conducteur(idConducteur),
  FOREIGN KEY (idPlace) REFERENCES Place(idPlace),
  FOREIGN KEY (idTarif) REFERENCES Tarif(idTarif)
);

-- Paiement
CREATE TABLE IF NOT EXISTS Paiement (
  idPaiement INT AUTO_INCREMENT PRIMARY KEY,
  idReservation INT UNIQUE NOT NULL,
  idSysteme INT NOT NULL,
  montant DECIMAL(10,2) NOT NULL,
  dateP TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  methodePaiement VARCHAR(50),
  statut ENUM('pending','successful','failed','refunded') DEFAULT 'successful',
  FOREIGN KEY (idReservation) REFERENCES Reservation(idReservation) ON DELETE
CASCADE,
  FOREIGN KEY (idSysteme) REFERENCES SystemePaiement(idSysteme)
);

-- Etablissement (GPS - parkings)
CREATE TABLE IF NOT EXISTS Etablissement (
  idEtablissement INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  adresse VARCHAR(255) NOT NULL,
  latitude DECIMAL(10,7) NOT NULL,
  longitude DECIMAL(10,7) NOT NULL,
  placesDisponibles INT DEFAULT 0
);

-- Notification
CREATE TABLE IF NOT EXISTS Notification (
  idNotification INT AUTO_INCREMENT PRIMARY KEY,
  idConducteur INT,
  message TEXT NOT NULL,
  dateNotification TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```sql
  typeNotification VARCHAR(50),
  lu TINYINT DEFAULT 0,
  FOREIGN KEY (idConducteur) REFERENCES Conducteur(idConducteur)
);

-- Données initiales - Moyens de paiement Djibouti
INSERT INTO SystemePaiement (typeSysteme) VALUES
  ('D-Money'), ('MStarCard'), ('Espèces'), ('Carte bancaire');

INSERT INTO Tarif (description, montantHoraire, montantJournalier) VALUES
  ('Tarif standard - MOUBARIK Djibouti', 500, 3000);

-- Admin : admin@moubarik.dj / password
INSERT INTO Administrateur (nom, email, motDePasse) VALUES
  ('Admin MOUBARIK', 'admin@moubarik.dj',
   '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi');

-- Conducteur : user@moubarik.dj / password
INSERT INTO Conducteur (nom, prenom, email, telephone, motDePasse) VALUES
  ('Demo', 'Utilisateur', 'user@moubarik.dj', '+253 77 00 00 00',
   '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi');

-- Places (45 places : RDC, B1, B2)
INSERT INTO Place (numero, localisation, statut, idTarif) VALUES
  ('RDC-A1','Rez-de-chaussee Zone A','available',1),
  ('RDC-A2','Rez-de-chaussee Zone A','available',1),
  ('RDC-A3','Rez-de-chaussee Zone A','occupied',1),
  ('RDC-A4','Rez-de-chaussee Zone A','reserved',1),
  ('RDC-A5','Rez-de-chaussee Zone A','available',1),
  ('RDC-B1','Rez-de-chaussee Zone B','occupied',1),
  ('RDC-B2','Rez-de-chaussee Zone B','available',1),
  ('RDC-B3','Rez-de-chaussee Zone B','available',1),
  ('RDC-B4','Rez-de-chaussee Zone B','occupied',1),
  ('RDC-B5','Rez-de-chaussee Zone B','available',1),
  ('RDC-C1','Rez-de-chaussee Zone C','available',1),
  ('RDC-C2','Rez-de-chaussee Zone C','occupied',1),
  ('RDC-C3','Rez-de-chaussee Zone C','reserved',1),
```

```
  ('RDC-C4','Rez-de-chaussee Zone C','available',1),
  ('RDC-C5','Rez-de-chaussee Zone C','occupied',1),
  ('B1-A1','Sous-sol 1 Zone A','available',1),
  ('B1-A2','Sous-sol 1 Zone A','occupied',1),
  ('B1-A3','Sous-sol 1 Zone A','reserved',1),
  ('B1-A4','Sous-sol 1 Zone A','available',1),
  ('B1-A5','Sous-sol 1 Zone A','occupied',1),
  ('B1-B1','Sous-sol 1 Zone B','available',1),
  ('B1-B2','Sous-sol 1 Zone B','available',1),
  ('B1-B3','Sous-sol 1 Zone B','occupied',1),
  ('B1-B4','Sous-sol 1 Zone B','available',1),
  ('B1-B5','Sous-sol 1 Zone B','occupied',1),
  ('B1-C1','Sous-sol 1 Zone C','occupied',1),
  ('B1-C2','Sous-sol 1 Zone C','available',1),
  ('B1-C3','Sous-sol 1 Zone C','available',1),
  ('B1-C4','Sous-sol 1 Zone C','occupied',1),
  ('B1-C5','Sous-sol 1 Zone C','available',1),
  ('B2-A1','Sous-sol 2 Zone A','available',1),
  ('B2-A2','Sous-sol 2 Zone A','occupied',1),
  ('B2-A3','Sous-sol 2 Zone A','available',1),
  ('B2-A4','Sous-sol 2 Zone A','reserved',1),
  ('B2-A5','Sous-sol 2 Zone A','available',1),
  ('B2-B1','Sous-sol 2 Zone B','occupied',1),
  ('B2-B2','Sous-sol 2 Zone B','occupied',1),
  ('B2-B3','Sous-sol 2 Zone B','available',1),
  ('B2-B4','Sous-sol 2 Zone B','available',1),
  ('B2-B5','Sous-sol 2 Zone B','occupied',1),
  ('B2-C1','Sous-sol 2 Zone C','available',1),
  ('B2-C2','Sous-sol 2 Zone C','occupied',1),
  ('B2-C3','Sous-sol 2 Zone C','available',1),
  ('B2-C4','Sous-sol 2 Zone C','available',1),
  ('B2-C5','Sous-sol 2 Zone C','occupied',1);


-- Etablissements (parkings avec coordonnées GPS Djibouti)
INSERT INTO Etablissement (nom, adresse, latitude, longitude, placesDisponibles)
VALUES
  ('MOUBARIK Centre','Place du 27 Juin, Djibouti', 11.5890, 43.1450, 45),
```

```
  ('MOUBARIK Ambouli','Avenue de l Aeroport, Djibouti', 11.5590, 43.1390, 30),
  ('MOUBARIK PK12','Route de Dikhil, Djibouti', 11.5240, 43.0780, 25);


-- Capteurs (un capteur par place)
INSERT INTO Capteur (idPlace, localisation, etat)
SELECT idPlace, localisation,
  IF(statut = 'available', 'empty', 'occupied') FROM Place;
```

## Annexe C